Department of Mathematics and Computer Science
University of Southern Denmark, Odense

29 May 2026

Simon Holm, Johannes Rothe

AI505 – Optimization

Answers to Obligatory Assignment 2, Spring 2026

Contents

Case 1

Let

$$P = \left\{x \in \mathbb{R}^n \mid \boldsymbol{a}_i^T x \leq b_i, i = 1, ..., m\right\} \tag{1}$$

be a nonempty, bounded polyhedron with nonempty interior.

Task 1 - Defining the center

The center $x^* \in P$ is defined by creating barrier functions that penalize by getting close the "sides" of the polyhedron, and then minimizing over $\boldsymbol{x}$: This is done by rewriting the linear constraint $\boldsymbol{a}_i^T \boldsymbol{x} \leq b_i$ to $s_i = \boldsymbol{a}_i^T \boldsymbol{x} - b_i$ such that the variable s_i describes the slack, or distance to the barrier. This can be made into a function whose output is the $\boldsymbol{x}$ "closeness" to violating each constraint:

$$g_i(\boldsymbol{x}) = \boldsymbol{a}_i^T \boldsymbol{x} - b_i \tag{2}$$

This is then turned into a log barrier, such that the closer the point is to the constraint, the penalty gets exponentially larger, allowing for a complete unconstrained problem:

$$\begin{aligned} &\min_{\boldsymbol{x} \in \mathbb{R}^n} f(\boldsymbol{x}) \\ &f(\boldsymbol{x}) = -\sum_{i=1}^{m} \log(-g_i(\boldsymbol{x})) \\ &g_i(\boldsymbol{x}) = \boldsymbol{a}_i^T \boldsymbol{x} - b_i \end{aligned} \tag{3}$$

This minimization satisfies all the given criterium of:

1. It lies strictly within P, since the closer the inner term gets to zero, (i.e the slack value decreases) the penalty explodes towards infinity because of the log barrier, and at last becomes undefined at 0^+. The domain of the function therefore needs to be:

$$\boldsymbol{x} \in D \quad D = \{\boldsymbol{x} \in \mathbb{R}^n : g_i(\boldsymbol{x}) < 0 \text{ for } \ i = 1, ..., m\} \tag{4}$$

1. It favors all the constraints equally (none are controlled with a constant) and the minimizing of $\boldsymbol{x}$ will find the point that minimizes *all* the constraints, i.e. gets as far away from all the sides as possible, ending in the middle of the polyhedron.
2. The log function is continuos on its domain.

Task 2 - Analytical properties

Computing the gradiens and the hessian

To compute $\nabla f(\boldsymbol{x})$, the function $f(\boldsymbol{x}) = -\sum_i \log(-g_i(\boldsymbol{x}))$ will be differentiated using the chain rule. Since the sum element of the function can just be viewed as a repeated application of the sum rule, differentiating the term in the sum, replaces all terms in the sum. The $\log(-g_i(\boldsymbol{x}))$ term can be differentiated with the chain rule:

$$
\begin{aligned}
\phi(\boldsymbol{x}) &= -g_i(\boldsymbol{x}) = -\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right) \\
z &= \phi(\boldsymbol{x}) \\
\sigma(z) &= \log(z) \\
\frac{\partial}{\partial \boldsymbol{x}} \phi(\boldsymbol{x}) &= -\boldsymbol{a}_i \\
\frac{\mathrm{d}}{\mathrm{d}z} \sigma(z) &= \frac{1}{z} \\
\frac{\partial}{\partial \boldsymbol{x}} \sigma(\phi(\boldsymbol{x})) &= \frac{-\boldsymbol{a}_i}{-\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)} = \frac{\boldsymbol{a}_i}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)} = \boldsymbol{a}_i \left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^{-1} \\
\nabla f(\boldsymbol{x}) &= -\sum_{i=1}^{m} \boldsymbol{a}_i \left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^{-1}
\end{aligned}
\tag{5}
$$

This describes the gradient of the function $f(\boldsymbol{x})$, i.e the change for each x value in $\boldsymbol{x}$. The hessian will then be the change for each x, if it is differentiated with either itself, or another $x \in \boldsymbol{x}$:

$$
\nabla^2 f(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}
\tag{6}
$$

Therefore the function can be differentiated again using the chain rule:

$$
\begin{aligned}
\phi(\boldsymbol{x}) &= \left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right) \\
z &= \phi(\boldsymbol{x}) \\
\sigma(z) &= (z)^{-1} \\
\frac{\partial}{\partial z}\sigma(z) &= -1(z)^{-2} && \text{| Outer term} \\
\frac{\partial}{\partial \boldsymbol{x}}\phi(\boldsymbol{x}) &= \boldsymbol{a}_i^T && \text{| Inner term} \\
\frac{\partial}{\partial \boldsymbol{x}}\sigma(\phi(\boldsymbol{x})) &= \sum_{i=1}^{m} \frac{\boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2} \\
&= \sum_{i=1}^{m} \frac{\boldsymbol{a}_i \boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2} && \text{| Multiply the } \boldsymbol{a_i} \text{ constant}
\end{aligned}
\tag{7}
$$

Meaning:

$$
\nabla^2 f(\boldsymbol{x}) = \sum_{i=1}^{m} \frac{\boldsymbol{a}_i \boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2} \tag{8}
$$

Proving convexity

By proving that the function is convex and suitable for second order methods, all the requirements of task 1 will be satisfied.

Using affine and convex properties

The function $-\sum_{i=1}^{m} \log\left(-\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)$ can be split into a sum with non-negative scalar, an inner affine function and an outer convex function.

$$\begin{aligned} -\sum_{i=1}^{m} \phi_i(\boldsymbol{x}) &= \sum_{i=1}^{m} 1 \cdot (-\phi_i(\boldsymbol{x})) && \text{Non negative scalar} \\ \sigma_i(\boldsymbol{x}) &= -\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right) && \text{Affine} \\ \phi_i(\boldsymbol{x}) &= -\log(\sigma_i(\boldsymbol{x})) && \text{Convex} \end{aligned} \tag{9}$$

By the following propositions;

Proposition: Convexity under sums

If $\{f_i\}_{j\in\{1,\dots,m\}}$ are convex and $\alpha_{j_{j\in\{1,\dots,m\}}}$ is non-negative, then $\sum_{j=1}^{m} \alpha_j f_j$ and $\max_{j\in\{1,\dots,m\}} f_j$ are convex

Proposition: Convexity on composition

If $f : \mathbb{R}^d \to \mathbb{R}$ is convex and $A : \mathbb{R}^d \to \mathbb{R}^d$ is affine then $f \circ A : \mathbb{R}^d \to \mathbb{R}$ is convex

And since σ is affine, and $\log(x)$ is convex, $f(x) = -\sum \phi$ is also convex. This can be seen since the function is a combination of a convex function $\log(x)$ (log function are concave, but since the entire function is a sum of these, with a negation in front, they become convex) and affine function $\boldsymbol{a}_i^T \boldsymbol{x} - b_i$.

Using semi-positive definiteness

It could also be argued that if the hessian is semi-positive definite that it is convex (WrightS 2006, p. 30):

$$\nabla^2 f(\boldsymbol{x}) \succeq 0 \quad \forall \boldsymbol{x} \in D \tag{10}$$

Where D needs to be a convex set. Since the inequality constrains can be defined as half-spaces, the intersections of the half-spaces forms a convex set as shown on Figure 1.

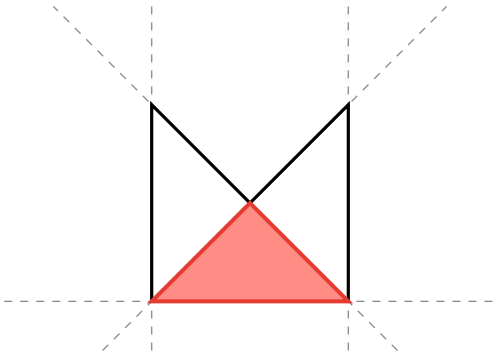

Figure 1: For any concave polygon (black) with infinite lines (half planes). Half-plane intersection (red) is always convex.

The rest can be found by looking at the hessian, and proving that all the entries have to be positive. A matrix is semi-positive definite if it is square and symmetric (which the hessian is by definition) and:

$$\boldsymbol{v}^T A \boldsymbol{v} \geq 0 \text{ for all } \boldsymbol{v} \neq 0 \tag{11}$$

Looking at the hessian:

$$\sum_{i=1}^{m} \frac{\boldsymbol{a}_i \boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2} \tag{12}$$

It can be observed that the bottom element is squared, thereby guaranteeing positivity. Therefore this can be rewritten as a constant that is multiplied to the other term:

$$\sum_{i=1}^{m} c_i \left(\boldsymbol{a}_i \boldsymbol{a}_i^T\right) \tag{13}$$

Since $\boldsymbol{a}_i\boldsymbol{a}_i^T \in \mathbb{R}^{n\times n}$, is a matrix, the positive semi-definite property can be tested:

$$\begin{aligned}
&\sum_{i=1}^{m} c_i\left(\boldsymbol{v}^T\boldsymbol{a}_i\boldsymbol{a}_i^T\boldsymbol{v}\right) \\
&\sum_{i=1}^{m} c_i\left((\boldsymbol{v}^T\boldsymbol{a}_i)(\boldsymbol{a}_i^T\boldsymbol{v})\right) \quad \text{The dot product is symmetric} \\
&\sum_{i=1}^{m} c_i(k_i)^2 \geq 0
\end{aligned} \tag{14}$$

Which proves that the function is positive semidefinte, and thereby convex.

Minimizer

A local minimizer can be defined as:

Definition: local minimizer

A point $\boldsymbol{x}^*$ is a *local minimum* (or is a local minimizer) if there exists a $\delta > 0$ such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$ with $\|\boldsymbol{x} - \boldsymbol{x}^*\| < \delta$.

Which means that for a specific point $\boldsymbol{x}^*$within a region, $f(\boldsymbol{x}^*)$ is smaller than all other $f(\boldsymbol{x})$. This can be argued trough an understanding of the properties of the function. First, the function is continuos and convex, meaning it's overall shape is that of a ball. The domain is strictly bounded within this "bowl", and the function value goes to infinity as the point moves closer to the edge of the "bowl". It can be further argued that this is a global minimizer, not only from the "bowl" analogy, but trough properties. If the previous proposition of a local minimizer is accepted, the following definition:

Definition: Local Minimizers in Convex Function

Assume that $f : \mathbb{R}^n \to \mathbb{R}$ is convex, then $\boldsymbol{x}^*$ is a global minimizer of f if and only if it is a local minimizer. If in addition f is differentiable, then $\boldsymbol{x}^*$ is a global minimizer of f if and only if it is a stationary point of f, ie., $\nabla f(\boldsymbol{x}^*) = 0$

Using this property, any local minimizer of a convex function, is a global minimizer.

Task 3

Task 3.1

Task 3.1 will first be solved explicitly, and thereafter a python program will be constructed to solve the problem iteratively. The problem can be formulated:

$$
\begin{aligned}
f(\boldsymbol{x}) &= -\sum_{i}^{5} \log(-g_i(\boldsymbol{x})) \\
&\text{Where } g_i(\boldsymbol{x}): \\
&\qquad g_1(\boldsymbol{x}) = 1x_1 + 0x_2 - 1 \\
&\qquad g_2(\boldsymbol{x}) = 0x_1 + 1x_2 - 1 \\
&\qquad g_3(\boldsymbol{x}) = (-1)x_1 + 0x_2 - 1 \\
&\qquad g_4(\boldsymbol{x}) = 0x_1 - 1x_2 - 1 \\
&\qquad g_5(\boldsymbol{x}) = 1x_1 + 1x_2 - 1.5
\end{aligned}
\tag{15}
$$

The Newton method for multivariate functions can be described in updates:

$$
\boldsymbol{x}_{i+1} = \boldsymbol{x}_i - \left(\nabla^2 f(\boldsymbol{x}_i)\right)^{-1} \nabla f(\boldsymbol{x}_i) \tag{16}
$$

Here, one stop will be done manually, and then rest of the iterative steps will be done using a python program. First, the hessian and inverse of the hessian can be calculated:

$$
\begin{gathered}
\sum_i \frac{\boldsymbol{a}_i \boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2} \\
\frac{\begin{bmatrix}1\\0\end{bmatrix} \cdot \begin{bmatrix}1 & 0\end{bmatrix}}{\left(\begin{bmatrix}1 & 0\end{bmatrix} \cdot \begin{bmatrix}0\\0\end{bmatrix} - 1\right)^2} = \begin{bmatrix}1 & 0\\0 & 0\end{bmatrix} \cdot \frac{1}{(-1)^2} = \begin{bmatrix}1 & 0\\0 & 0\end{bmatrix} \\
\frac{\begin{bmatrix}0\\1\end{bmatrix} \cdot \begin{bmatrix}0 & 1\end{bmatrix}}{\left(\begin{bmatrix}0 & 1\end{bmatrix} \cdot \begin{bmatrix}0\\0\end{bmatrix} - 1\right)^2} = \begin{bmatrix}0 & 0\\0 & 1\end{bmatrix} \cdot \frac{1}{(-1)^2} = \begin{bmatrix}0 & 0\\0 & 1\end{bmatrix} \\
\frac{\begin{bmatrix}-1\\0\end{bmatrix} \cdot \begin{bmatrix}-1 & 0\end{bmatrix}}{\left(\begin{bmatrix}-1 & 0\end{bmatrix} \cdot \begin{bmatrix}0\\0\end{bmatrix} - 1\right)^2} = \begin{bmatrix}1 & 0\\0 & 0\end{bmatrix} \cdot \frac{1}{(-1)^2} = \begin{bmatrix}1 & 0\\0 & 0\end{bmatrix} \\
\frac{\begin{bmatrix}0\\-1\end{bmatrix} \cdot \begin{bmatrix}0 & -1\end{bmatrix}}{\left(\begin{bmatrix}0 & -1\end{bmatrix} \cdot \begin{bmatrix}0\\0\end{bmatrix} - 1\right)^2} = \begin{bmatrix}0 & 0\\0 & 1\end{bmatrix} \cdot \frac{1}{(-1)^2} = \begin{bmatrix}0 & 0\\0 & 1\end{bmatrix} \\
\frac{\begin{bmatrix}1\\1\end{bmatrix} \cdot \begin{bmatrix}1 & 1\end{bmatrix}}{\left(\begin{bmatrix}1 & 1\end{bmatrix} \cdot \begin{bmatrix}0\\0\end{bmatrix} - 1.5\right)^2} = \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix} \cdot \frac{1}{(-1.5)^2} = \begin{bmatrix}\frac{1}{(-1.5)^2} & \frac{1}{(-1.5)^2}\\\frac{1}{(-1.5)^2} & \frac{1}{(-1.5)^2}\end{bmatrix} \\
\begin{bmatrix}1 & 0\\0 & 0\end{bmatrix} + \begin{bmatrix}0 & 0\\0 & 1\end{bmatrix} + \begin{bmatrix}1 & 0\\0 & 0\end{bmatrix} + \begin{bmatrix}0 & 0\\0 & 1\end{bmatrix} + \begin{bmatrix}\frac{1}{(-1.5)^2} & \frac{1}{(-1.5)^2}\\\frac{1}{(-1.5)^2} & \frac{1}{(-1.5)^2}\end{bmatrix} = \begin{bmatrix}2.44 & 0.44\\0.44 & 2.44\end{bmatrix} \\
\left(\nabla^2 f\right)^{-1} = \frac{1}{(2.44 \cdot 2.44) - (0.44 \cdot 0.44)} \begin{bmatrix}2.44 & -0.44\\-0.44 & 2.44\end{bmatrix} = \begin{bmatrix}0.41 & -0.07\\-0.07 & 0.41\end{bmatrix}
\end{gathered}
\tag{17}
$$

And the gradient:

$$
\begin{gathered}
-\sum_{i=1}^{m} \frac{\boldsymbol{a}_i}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)} \\
-\left(\begin{bmatrix}-1\\0\end{bmatrix} + \begin{bmatrix}0\\-1\end{bmatrix} + \begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\1\end{bmatrix} + \begin{bmatrix}-0.67\\-0.67\end{bmatrix}\right) = \begin{bmatrix}0.67\\0.67\end{bmatrix}
\end{gathered}
\tag{18}
$$

And then the newton step:

$$
\boldsymbol{x}_{\text{new}} = \begin{bmatrix}0\\0\end{bmatrix} - \begin{bmatrix}0.41 & -0.07\\-0.07 & 0.41\end{bmatrix} \cdot \begin{bmatrix}0.67\\0.67\end{bmatrix} = \begin{bmatrix}-0.23\\-0.23\end{bmatrix} \tag{19}
$$

The slack for this point would be:

$$\begin{aligned}
g_1(\boldsymbol{x}) &= 1x_1 + 0x_2 - 1 = (-0.23) - 1 = -1.23 \\
g_2(\boldsymbol{x}) &= 0x_1 + 1x_2 - 1 = (-0.23) - 1 = -1.23 \\
g_3(\boldsymbol{x}) &= (-1)x_1 + 0x_2 - 1 = 0.23 - 1 = -0.77 \\
g_4(\boldsymbol{x}) &= 0x_1 + (-1)x_2 - 1 = 0.23 - 1 = -0.77 \\
g_5(\boldsymbol{x}) &= 1x_1 + 1x_2 - 1.5 = (-0.23) + (-0.23) - 1.5 = -1.96
\end{aligned} \tag{20}$$

Since the slack is the distance to each barrier, to get the minimum slack, the absolute value of the smallest value is the minimum slack:

$$|-0.77| = 0.77 \tag{21}$$

This can also be expressed in code (Code snippet 1), using autograd to compute the gradient and hessian:

```python
def obj_func(x, order=1):
    def f(x):
        return -(
            anp.log( -(x[0] * 1 + x[1] * 0 - 1) ) +
            anp.log( -(x[0] * 0 + x[1] * 1 - 1) ) +
            anp.log( -(x[0] * (-1) + x[1] * 0 - 1) ) +
            anp.log( -(x[0] * 0 - x[1] * 1 - 1) ) +
            anp.log( -(x[0] * 1 + x[1] * 1 - 1.5) )
            )

    def slack(x):
        return anp.array(
            [(x[0] * 1 + x[1] * 0 - 1),
            (x[0] * 0 + x[1] * 1 - 1),
            (x[0] * (-1) + x[1] * 0 - 1),
            (x[0] * 0 - x[1] * 1 - 1),
            (x[0] * 1 + x[1] * 1 - 1.5)]
        )

    match order:
        case 0:
            return slack(x), f(x)
        case 1:
            return slack(x), f(x), grad(f)(x)
        case 2:
            return slack(x), f(x), grad(f)(x), jacobian(grad(f))(x)
```

Code snippet 1: The objective function defining both the objective and an array of the slacks of each inequality constraints. It return the slack $s(x)$, the gradient ∇f and the hessian $\nabla^2 f$

That one can then use the newton method as shown on Code snippet 2:

```python
def newton_method(x):
    _, _, grad, hes = obj_func(x, order=2)
    return x - anp.linalg.inv(hes) @ grad
```

Code snippet 2: One step of the newton method

Using this to test the point $\boldsymbol{x} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ yields:

```
output
Hessian for original x: [[2.44444444 0.44444444]
 [0.44444444 2.44444444]]
New x after newton method: [-0.23076923 -0.23076923]
Obj value for x with newton method: -0.5642792953243507
Slack for x with newton method: [-1.23076923 -1.23076923 -0.76923077
-0.76923077 -1.96153846]
```

Code snippet 3: Output after 1 iteration/step of the newton method as implemented on Code snippet 2

Which matches the earlier calculations. The original point in red, and the new point in green can then be visualized (Figure 2):

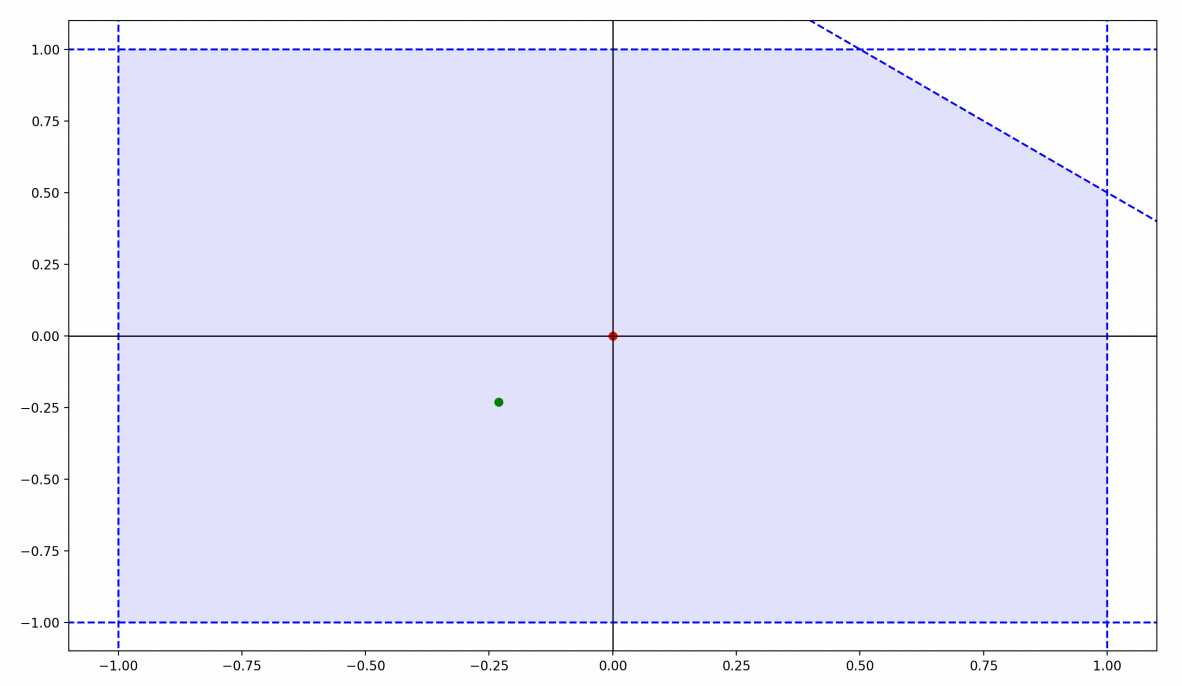

Figure 2: The feasible region shown in blue, with the original $\boldsymbol{x}$ (red) and updated $\boldsymbol{x}$ (green)

But since just doing one iteration is not guaranteed to be the center point, the newton method can be modified to incorporate stopping criterions, such as max iterations or $|f(\boldsymbol{x_{i+1}}) - f(\boldsymbol{x_i})| < \delta$:

Input: $\nabla f, H, \mathbf{x}_0, \epsilon, k_{max}$
Output: $\mathbf{x}^*$
Set $k = 0, \Delta = 1, \mathbf{x} = \mathbf{x}_0$;
while $\|\Delta\| > \epsilon$ and $k \leq k_{max}$ **do**
    $\Delta = H(\mathbf{x})^{-1} \nabla f(\mathbf{x})$;
    $\mathbf{x} = \mathbf{x} - \Delta$;
    $k = k + 1$;

Figure 3: The Newton Method algorithm in pseudocode

```python
def newton_method_stop(x, crit, max_iter):
    i = 0
    while i < max_iter :
        _, _, grad, hes = obj_func(x, order=2)
        x_new = x - anp.linalg.inv(hes) @ grad
        if math.dist(x_new, x) <= crit:
            return x_new
        else:
            x = x_new
        i += 1
    return x
```

Code snippet 4: Newton method implemented as from Figure 3

Running this (Code snippet 4) with `max_iter=200` and `crit=0.001` yields:

```output
New x after newton method with stop: [-0.23851648 -0.23851648]
Obj value for x with newton method with stop: -0.5644522715736566
Slack for x after newton method with stop: [-1.23851648 -1.23851648 -0.
76148352 -0.76148352 -1.97703296]

Distance from x after first itertaion, to after full newton method:
0.01095626629433487
Differece in objective value after first x, to after full newton method:
0.0001729762493059006
```

Code snippet 5: Output after the iterated newton method (Code snippet 4)

Here the center point is found to be

$$\boldsymbol{x}^* = \texttt{[-0.23851648 -0.23851648]} \tag{22}$$

with the minimum slack being =

$$|-0.76148352| = 0.76148352. \tag{23}$$

This can then be visualized aswell:

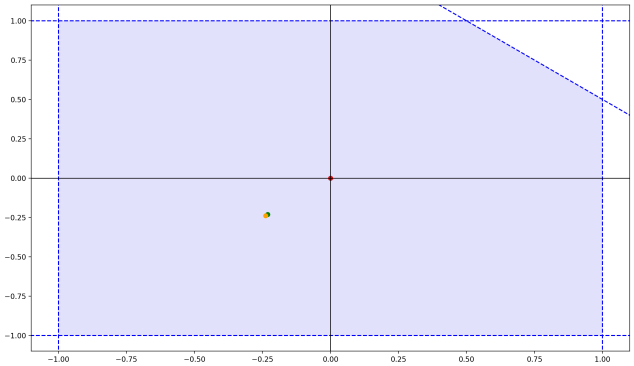

Figure 4: Visualization of the new center after iterative Newton Method (Code snippet 4)

And zoomed in:

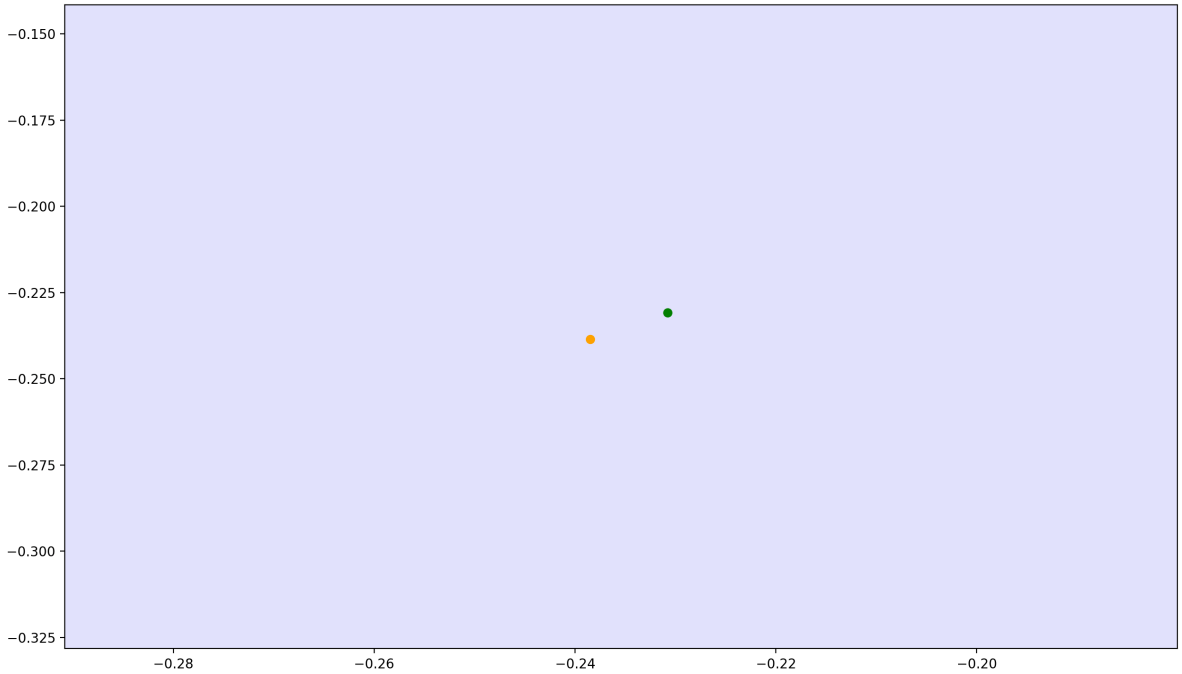

Figure 5: Zooming closer to the new center-point from Figure 4

Task 3.2

The change imposed on the last constraint can be expressed as:

$$\begin{aligned} f(\boldsymbol{x}) &= \left(-\sum_{i}^{4} \log(-g_i(\boldsymbol{x})) \right) - \log\big(-\big(\gamma \boldsymbol{a}_5^T \boldsymbol{x} - \gamma 1.5\big)\big) \\ &= \left(-\sum_{i}^{4} \log(-g_i(\boldsymbol{x})) \right) - \big(\log(\gamma) + \log\big(-\big(\boldsymbol{a}_5^T \boldsymbol{x} - 1.5\big)\big)\big) \end{aligned} \tag{24}$$

And since $\log(\gamma)$ is a constant, it disappears when we take the derivative, leaving the feasible and solution set unchanged. This holds for all values of γ. The actual value of the objective function does change, since the γ values are still used in calculating them.

The code can be easily modified to take the γ into account by modifying the objective function:

```python
def obj_func_modified(x, order=1, gam=1):
    def f(x):
        return -(
            anp.log( -(x[0] * 1 + x[1] * 0 - 1) ) +
            anp.log( -(x[0] * 0 + x[1] * 1 - 1) ) +
            anp.log( -(x[0] * (-1) + x[1] * 0 - 1) ) +
            anp.log( -(x[0] * 0 - x[1] * 1 - 1) ) +
            anp.log( -(x[0] * (1*gam) + x[1] * (1*gam)
                    - (1.5*gam)))
            )
    def slack(x):
        return anp.array(
            [-(x[0] * 1 + x[1] * 0 - 1),
            -(x[0] * 0 + x[1] * 1 - 1),
            -(x[0] * (-1) + x[1] * 0 - 1),
            -(x[0] * 0 - x[1] * 1 - 1),
            -(x[0] * (1*gam) + x[1] * (1*gam) - (1.5*gam))]
        )

    match order:
        case 0:
            return slack(x), f(x)
        case 1:
            return slack(x), f(x), grad(f)(x)
        case 2:
            return slack(x), f(x), grad(f)(x), jacobian(grad(f))(x)
```

Code snippet 6: Objective function, with gamme highlighted in red

And executing this with $\gamma = 10$ yields:

```output
x with gam=10: [-0.23851648 -0.23851648]
Obj value for x with gam=10: -0.5644522715736567
Slack for x with gam=10: [-1.23851648 -1.23851648 -0.76148352
-0.76148352 -1.97703296]
Distance from x after first itertaion, to after full newton method:
7.850462293418876e-17
Differece in objective value after first x, to after full newton method:
-1.1102230246251565e-16
```

Code snippet 7: Output after the iterated newton method with gamma modification. (Code snippet 4)

Showing that the 2 points are identical (the only difference is computing noise python). This can also be visualized:

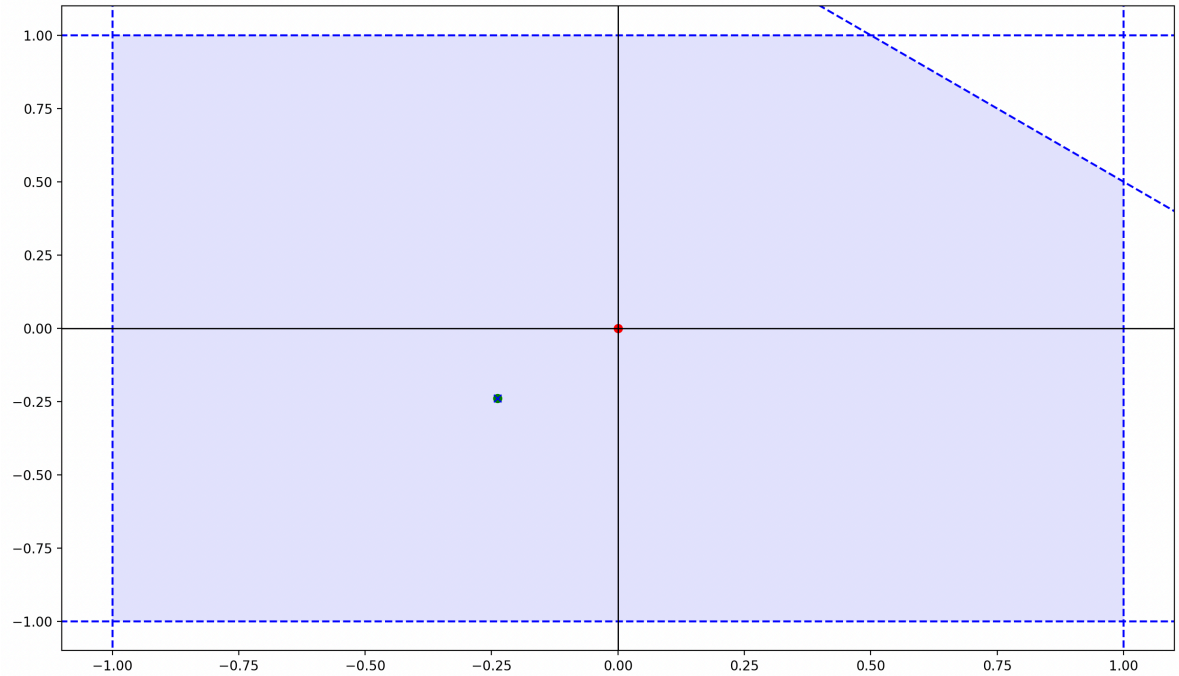

Figure 6: Newton method with $\gamma = 10$, the red dot being the starting point, green is after the newton method with no γ and blue $\times$ being the newton method with $\gamma = 10$.

And zooming in (a lot) does not change that they overlap:

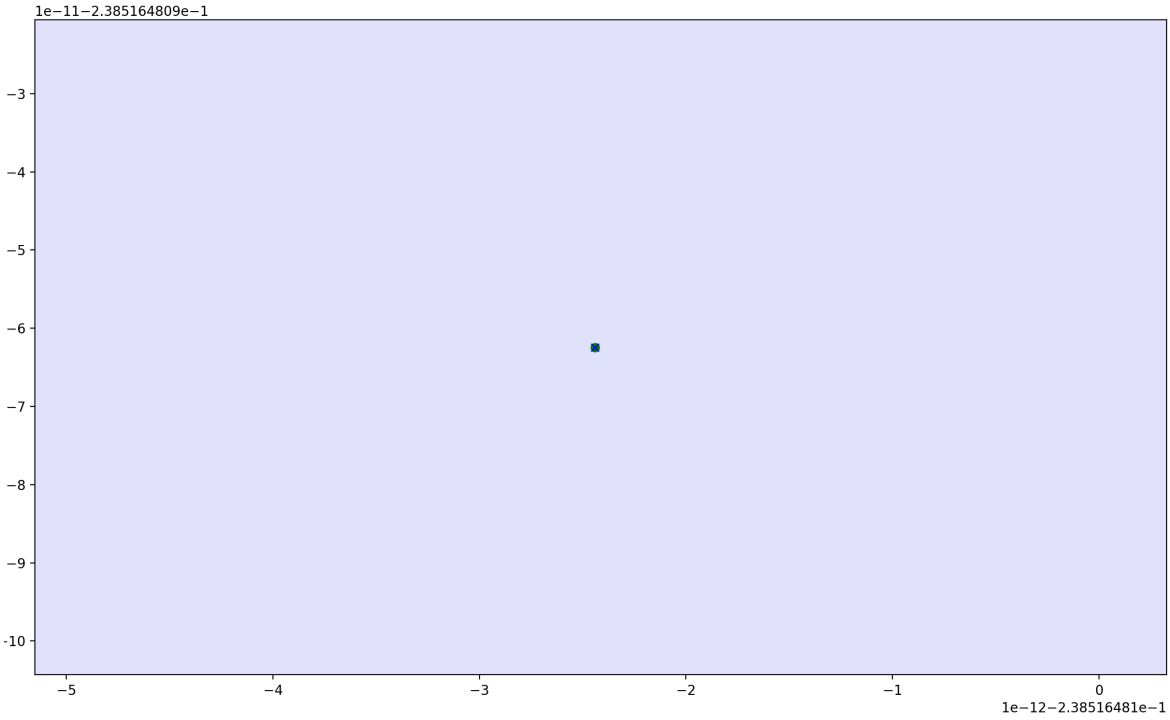

Figure 7: Newton method with $\gamma = 10$, green is after the newton method with no γ and blue $\times$ being the newton method with $\gamma = 10$.

This can then be done for all the given γ values, yielding the visualization:

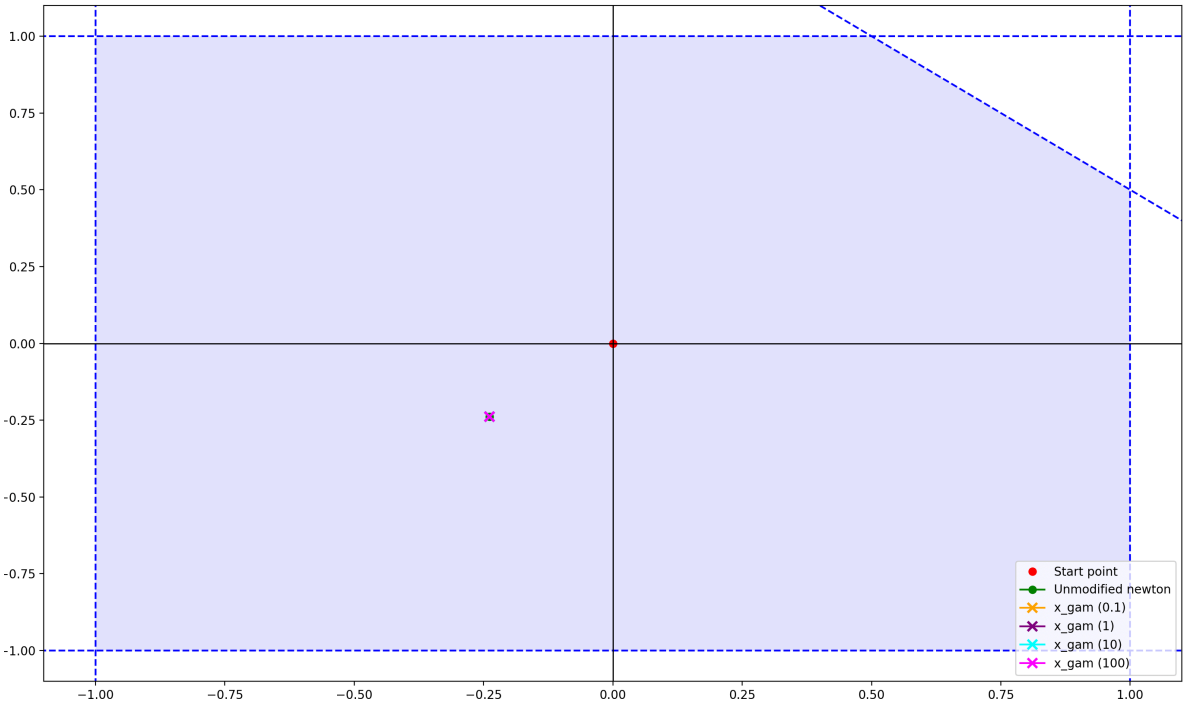

Figure 8: Visualization of different γ values.

This had the following output:

```
output
--- Results for gam=0.1 ---
x with gam=0.1: [-0.23851648 -0.23851648]
Obj value for x with gam=0.1: 1.7381328214203888
Distance to x_newton_stop: 7.850462293418876e-17
Difference in objective value from x_newton_stop: 2.3025850929940455

--- Results for gam=1 ---
x with gam=1: [-0.23851648 -0.23851648]
Obj value for x with gam=1: -0.5644522715736566
Distance to x_newton_stop: 0.0
Difference in objective value from x_newton_stop: 0.0

--- Results for gam=10 ---
x with gam=10: [-0.23851648 -0.23851648]
Obj value for x with gam=10: -2.8670373645677025
Distance to x_newton_stop: 7.850462293418876e-17
Difference in objective value from x_newton_stop: -2.302585092994046

--- Results for gam=100 ---
x with gam=100: [-0.23851648 -0.23851648]
Obj value for x with gam=100: -5.169622457561748
Distance to x_newton_stop: 3.925231146709438e-17
Difference in objective value from x_newton_stop: -4.605170185988092
```

Code snippet 8: Output after the iterated newton method with gamma modification, using different γ values. (Code snippet 4)

Task 3.3

Normalizing the last constraint is as simple as modifying the code:

```python
def obj_func_normalized(x, order=1, gam=1):
    def f(x):
        return -(
            anp.log( -(x[0] * 1 + x[1] * 0 - 1) ) +
            anp.log( -(x[0] * 0 + x[1] * 1 - 1) ) +
            anp.log( -(x[0] * (-1) + x[1] * 0 - 1) ) +
            anp.log( -(x[0] * 0 - x[1] * 1 - 1) ) +
            anp.log( -(x[0] * (1*gam) + x[1] * (1*gam) - (1*gam)) )
            )
    def slack(x):
        return anp.array(
            [-(x[0] * 1 + x[1] * 0 - 1),
            -(x[0] * 0 + x[1] * 1 - 1),
            -(x[0] * (-1) + x[1] * 0 - 1),
            -(x[0] * 0 - x[1] * 1 - 1),
            -(x[0] * (gam*1) + x[1] * (1*gam) - 1)]
        )

    match order:
        case 0:
            return slack(x), f(x)
        case 1:
            return slack(x), f(x), grad(f)(x)
        case 2:
            return slack(x), f(x), grad(f)(x), jacobian(grad(f))(x)
```

Code snippet 9: Objective function, with modifications highlighted in red

Where the difference in points can be recompupted:

```
--- Results for norm ---                                                    output
x with norm: [-0.28989802 -0.28989802]
Obj value for x with norm: -0.5567027826682145
Slack for x with norm: [-1.28989802 -1.28989802 -0.71010198 -0.71010198
-2.07979604]
Distance to x_newton_stop: 0.07266446749572646
Difference in objective value from x_newton_stop: 0.007749488905442137
```

Code snippet 10: Output of iterative newton method with normalized constraint.

And plotted:

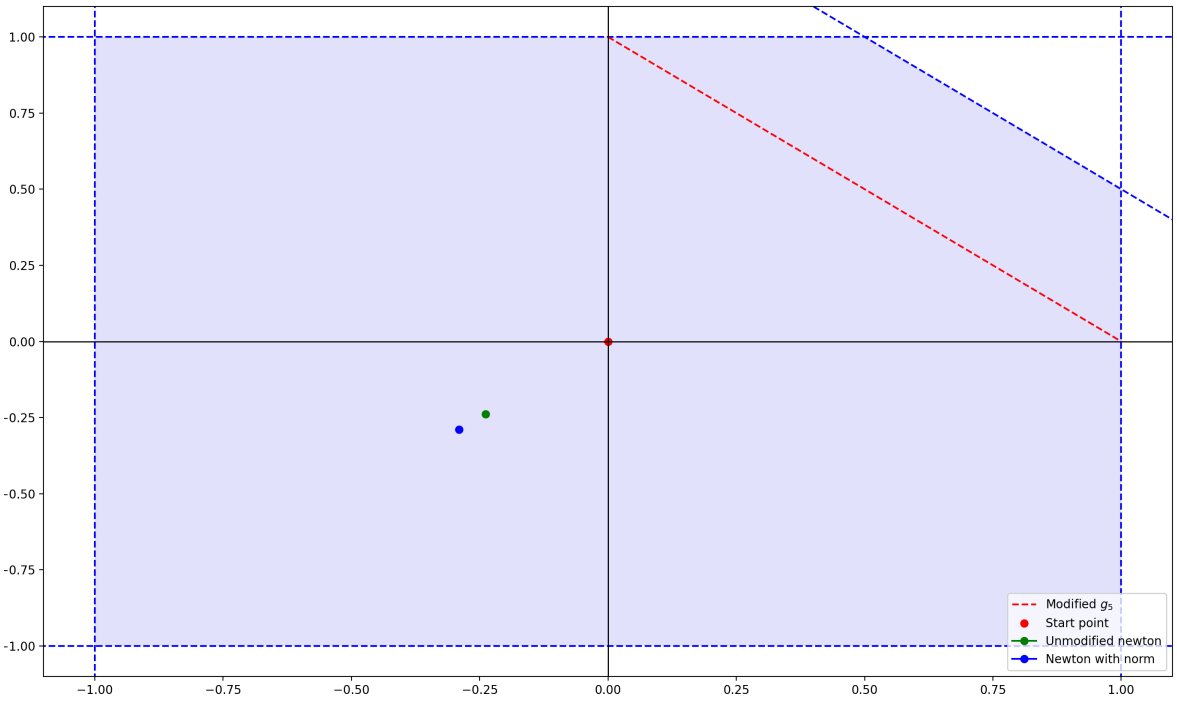

Figure 9: The modified feasible space, showing the red initial point, green point for newton method without modified constrains and the blue point for the modified constraint

Here the constraint has been moved, “cutting” a corner of the feasible space out. As the optimization algorithm finds the center of a given feasible space, it will move the center away from the area where the cutout is. This can also be explained as the optimization algorithm tries to balance the distance to each side of the feasible space, and if one of them moves closer, the point should then move accordingly.

Task 4

The problem can be formulated as follows:

Let

$$P = \left\{ \boldsymbol{x} \in \mathbb{R}^n \mid \boldsymbol{a}_i^T \boldsymbol{x} \leq b_i, -1 \leq x_j \leq 1, i = 1, ..., m, j = 1, ..., n \right\} \tag{25}$$

$$f(\boldsymbol{x}) = -\sum_{i=1}^{m} \log\left(-\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)\right) + \sum_{j=1}^{n} \left(-\log\left(x_j + 1\right)\right) + \sum_{j=1}^{n} \left(-\log\left(1 - x_j\right)\right) \tag{26}$$

By using the standard log barrier on the indivudal values of the vector $\boldsymbol{x}$, it is possibe make it into an unconstrained linear optimization problem (assuming that the initial point $\boldsymbol{x} \in D$, where D is the convex set).

By reusing the log barrier properties, each optimization problem of this form will simply treat the $|x_i| \leq 1$ as linear constraints.

Compute $\nabla f(\boldsymbol{x})$ and $\nabla^2 f(\boldsymbol{x})$

The first term has been found from task 2, the second and third term can be found using the chain rule:

$$\begin{gathered}
\sum_{j=1}^{n} \left(-\log\left(x_j + 1\right)\right) \\
\frac{\mathrm{d}}{\mathrm{d}x}[\log(x)] = \frac{1}{x} \\
\frac{\mathrm{d}}{\mathrm{d}x}\left[x_j + 1\right] = 1 \\
\frac{\mathrm{d}}{\mathrm{d}x}\left[\left(-\log\left(x_j + 1\right)\right)\right] = -\left(\frac{1}{x_j + 1}\right)
\end{gathered} \tag{27}$$

$$\begin{gathered}
\sum_{j=1}^{n} \left(-\log\left(1 - x_j\right)\right) \\
\frac{\mathrm{d}}{\mathrm{d}x}[\log(x)] = \frac{1}{x} \\
\frac{\mathrm{d}}{\mathrm{d}x}\left[1 - x_j\right] = -1 \\
\frac{\mathrm{d}}{\mathrm{d}x}\left[\left(-\log\left(1 - x_j\right)\right)\right] = -\left(\frac{-1}{1 - x_j}\right) = \sum_{j=1}^{n} \left(\frac{1}{1 - x_j}\right)
\end{gathered} \tag{28}$$

$$\nabla f(\boldsymbol{x}) = -\sum_{i=1}^{m} \boldsymbol{a}_i \left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^{-1} - \sum_{j=1}^{n} \left(\frac{1}{x_j + 1}\right) + \sum_{j=1}^{n} \left(\frac{1}{1 - x_j}\right) \tag{29}$$

Since the gradient is a vector, its important that the final output should also be a vector. The sums currently evaluate to scalars, to fix this, the appropriate basis vector is multiplied for each sum term. Since the sum term calculates the gradient value, the e basis vector applies it to the correct element of the gradient vector:

$$\nabla f(\boldsymbol{x}) = -\sum_{i=1}^{m} \boldsymbol{a}_i \left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^{-1} - \sum_{j=1}^{n} \left(\frac{\boldsymbol{e_j}}{x_j + 1}\right) + \sum_{j=1}^{n} \left(\frac{\boldsymbol{e}_j}{1 - x_j}\right) \tag{30}$$

Now to get the hessian, the same logic is applied. The function gets differentiated:

$$\frac{\mathrm{d}}{\mathrm{d}x}\left[-\left(\frac{\boldsymbol{e}_j}{x_j + 1}\right)\right] = \left[-\boldsymbol{e}_j \left(x_j + 1\right)^{-1}\right] = \boldsymbol{e}_j \left(x_j + 1\right)^{-2} \tag{31}$$

$$\frac{\mathrm{d}}{\mathrm{d}x}\left[\frac{\boldsymbol{e}_j}{1 - x_j}\right] = \left[\boldsymbol{e}_j \left(1 - x_j\right)^{-1}\right] = -\boldsymbol{e}_j \left(1 - x_j\right)^{-2} \cdot (-1) = \boldsymbol{e}_j \left(1 - x_j\right)^{-2} \tag{32}$$

$$\nabla^2 f(\boldsymbol{x}) = \sum_{i=1}^{m} \left(\frac{\boldsymbol{a}_i \boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2}\right) + \sum_{j=1}^{n} \left(\frac{\boldsymbol{e}_j}{\left(x_j + 1\right)^2}\right) + \sum_{j=1}^{n} \left(\frac{\boldsymbol{e}_j}{\left(x_j - 1\right)^2}\right) \tag{33}$$

Much like before, now the hessian is a matrix, therefore the output should also be a matrix as well. To do this, another basis vector is multiplied for each sum term.

$$\nabla^2 f(\boldsymbol{x}) = \sum_{i=1}^{m} \left(\frac{\boldsymbol{a}_i \boldsymbol{a}_i^T}{\left(\boldsymbol{a}_i^T \boldsymbol{x} - b_i\right)^2}\right) + \sum_{j=1}^{n} \left(\frac{\boldsymbol{e}_j \boldsymbol{e}_j^T}{\left(x_j + 1\right)^2}\right) + \sum_{j=1}^{n} \left(\frac{\boldsymbol{e}_j \boldsymbol{e}_j^T}{\left(x_j - 1\right)^2}\right) \tag{34}$$

Here it can be reused that the first part is a positive matrix, and since the second and third term are sum of a positive integer divided by something squared, they're also positive. The expression can then be seen as $H + D_1 + D_2$ where they are all positive, proving that the hessian is positive semi-definite.

Prove that $f(\boldsymbol{x})$ is convex on its domain

Expanding on the proof of convexity, it is already clear that the first term is convex. Since the second term $\sum_{j=1}^{n}(-\log(x_j+1))$ and third term $\sum_{j=1}^{n}(-\log(1-x_j))$ are convex using the same logic, the new function is just sums of convex functions, and the proposition for convexity under sums still holds:

Proposition: Convexity under sums

If $\{f_i\}_{j\in\{1,\dots,m\}}$ are convex and $\alpha_{j_{j\in\{1,\dots,m\}}}$ is non-negative, then $\sum_{j=1}^{m}\alpha_j f_j$ and $\max_{j\in\{1,\dots,m\}} f_j$ are convex

This can also be proved using the property:

$$\nabla^2 f(\boldsymbol{x}) \succeq 0 \quad \forall \boldsymbol{x} \in D \tag{35}$$

From (WrightS 2006, p. 30), where D again needs to be a convex set. Using the same arguments as before, the inequality constriants can be seen as half spaces, and this makes the domain a convex set. And since it has been established that the $\nabla^2 f$ is semi-positive definate, it is also convex

Lets test this on a case similar to earlier (but in 3 dimensions!). For this one should ensure that the methods should use

- **Search direction**

This is the negative gradient, that is

$$-\nabla f(x) \quad \text{(steepest descent)} \tag{36}$$

- **Backtracking line search**

The step size α_k is chosen via strong bracketing, which enforces both the Armijo (sufficient decrease) condition:

$$f(\boldsymbol{x}+\alpha\boldsymbol{d}) \leq f(\boldsymbol{x}) + \beta\alpha\nabla f_{\boldsymbol{d}}(\boldsymbol{x}) \tag{37}$$

and the curvature (strong Wolfe) condition:

$$|\nabla f_{\boldsymbol{d}}(\boldsymbol{x}+\alpha\boldsymbol{d})| \leq \sigma\ |\nabla f_{\boldsymbol{d}}(\boldsymbol{x})| \tag{38}$$

- A **mechanism to ensure feasibility** (iterates remain in domain)

After each step, the iterate is clipped to remain strictly inside the box constraints:

$$\boldsymbol{x}_{k+1} = \text{clip}(\boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k, -1 + \varepsilon, 1 - \varepsilon) \tag{39}$$

Such that any overshooting stays within $|x| \leq 1$
- A **stopping criterion** based on optimality measures: i.e., $\|\nabla f(x)\|_2 \leq \eta$.

When the gradient norm $\|\nabla f\|$ is sufficiently small, the iterate is near a stationary point (local optima)

```python
if anp.linalg.norm(g) <= eta:
            break
```

Code snippet 11: Stopping criterion stopping the search when close to a local optima.

Lets try this out on simple gradient descent problem with only one other inequality constraint.

Lets

$$A = [0 \;\; 1 \;\; 1] \text{ and } b = [0] \quad x_0 = [0. \;\; -0.7 \;\; 0.7] \tag{40}$$

Where x_0 is the initial search point.

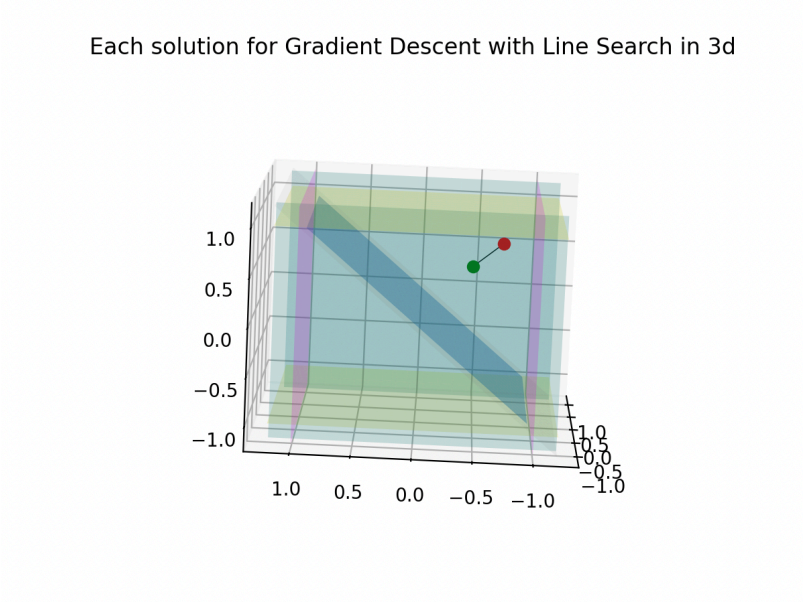

Figure 10: Gradient Descent using line search to find a center point given the barriers, $x \leq 1$ and $Ax \leq b$ from initial point x_0.
$x_{\text{best}} =$ `[ 0.  -0.4519704  0.4519704]`.

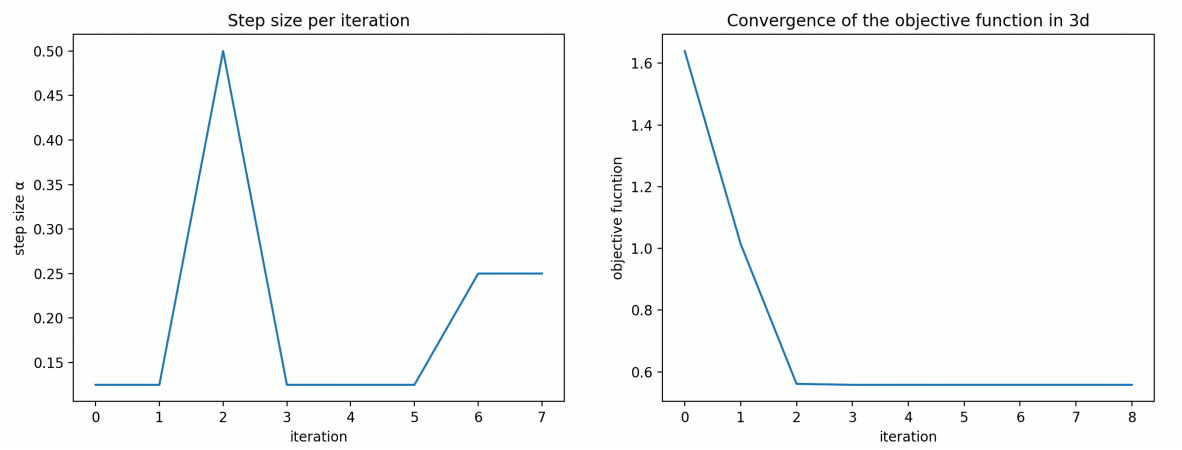

Figure 11: *Left*: Alpha returns during line-search gradient descent from Figure 10. *Right*: Convergence of the objective function from gradient descent from Figure 10

Lets add another inequality constraint

$$A = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \end{bmatrix} \text{ and } b = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad x_0 = [0. \ -0.7 \ \ 0.7] \tag{41}$$

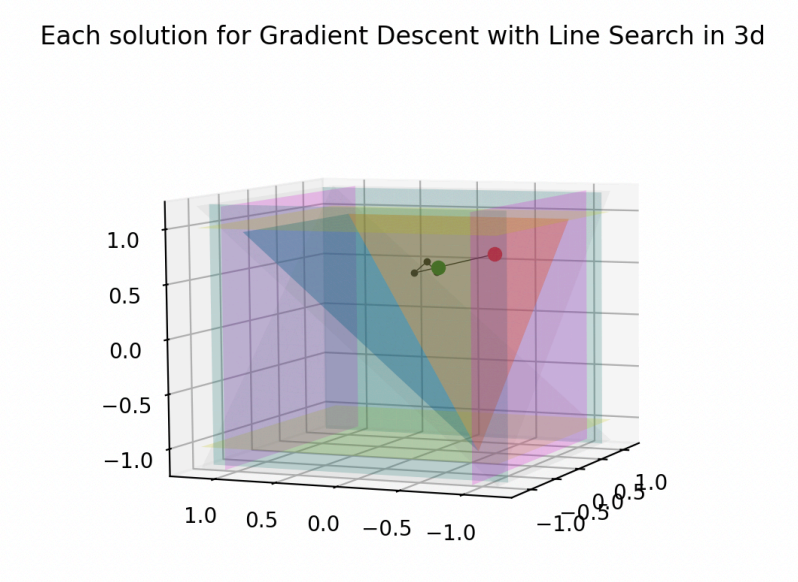

Figure 12: Gradient Descent using line search to find a center point given the barriers, $x \leq 1$ and $Ax \leq b$ from initial point x_0.
$x_{\text{best}} =$ `[-0.40824535 -0.40824813  0.61236766]`.

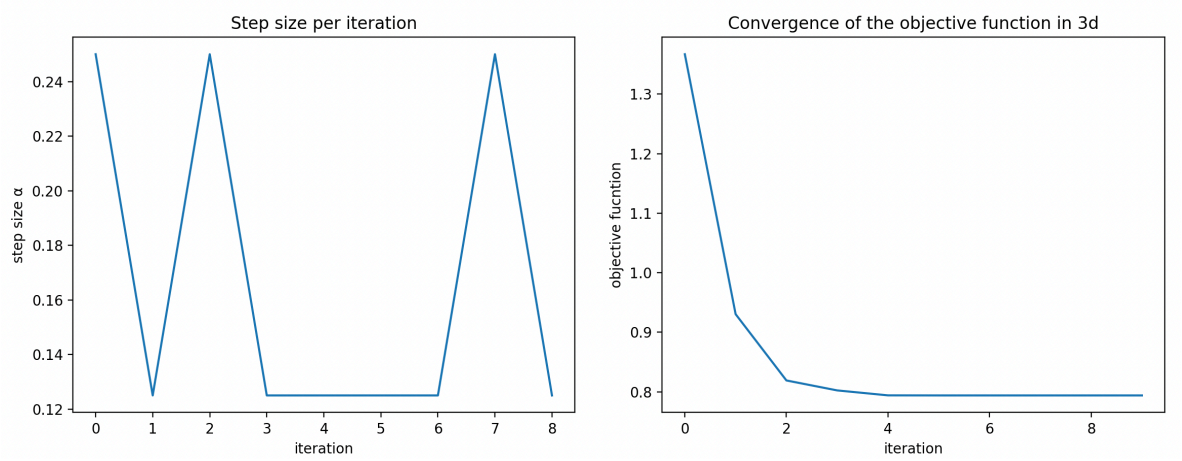

Figure 13: *Left*: Alpha returns during line-search gradient descent from Figure 12. *Right*: Convergence of the objective function from gradient descent from Figure 12

Lets try and apply the newton method.

Again let

$$A = [0 \;\; 1 \;\; 1] \text{ and } b = [0] \quad x_0 = [0. \;\; -0.7 \;\; 0.7] \tag{42}$$

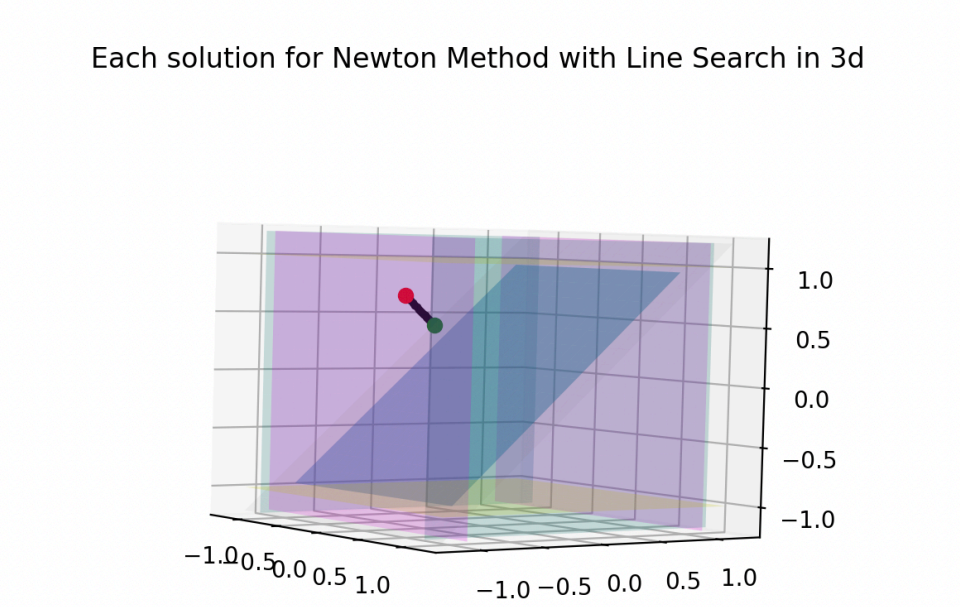

Figure 14: Newton method using line search to find a center point given the barriers, $x \leq 1$ and $Ax \leq b$ from initial point x_0.
$x_{\text{best}} =$ `[ 0.  -0.44720245  0.44720245]`.

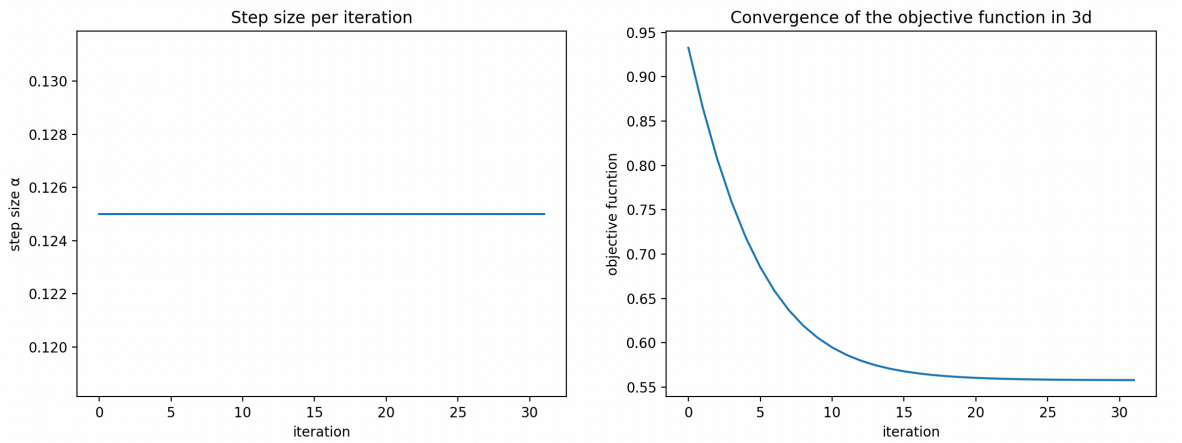

Figure 15: *Left*: Alpha returns during line-search newton method from Figure 14. *Right*: Convergence of the objective function from newton method from Figure 14

Now let

$$A = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \end{bmatrix} \text{ and } b = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad x_0 = [0. \ -0.7 \ \ 0.7] \tag{43}$$

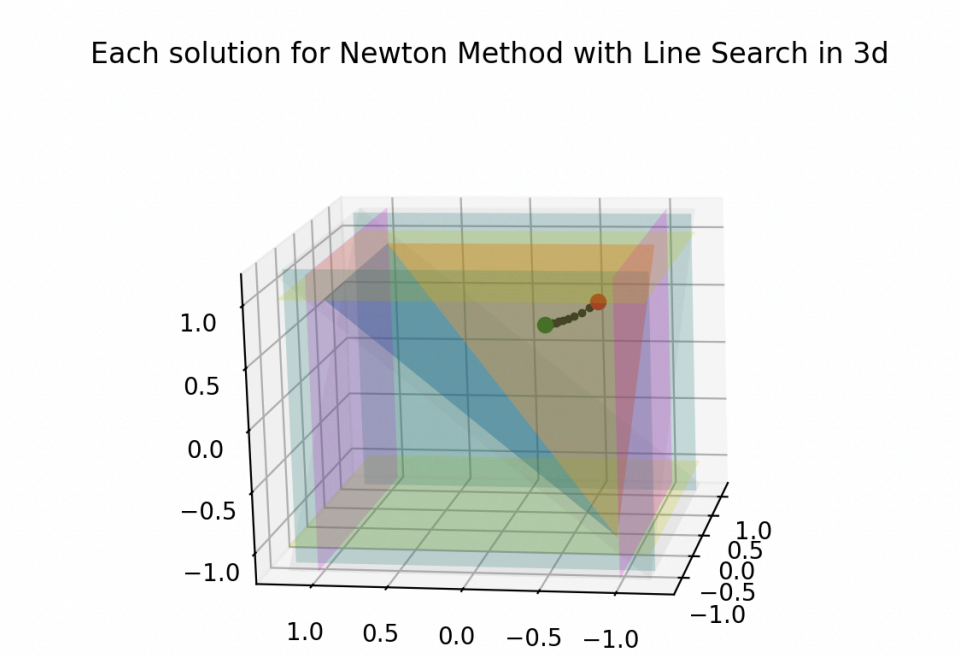

Figure 16: Newton method using line search to find a center point given the barriers, $x \leq 1$ and $Ax \leq b$ from initial point x_0.
$x_{\text{best}} =$ `[-0.40683268 -0.4105287  0.6125544 ]`.

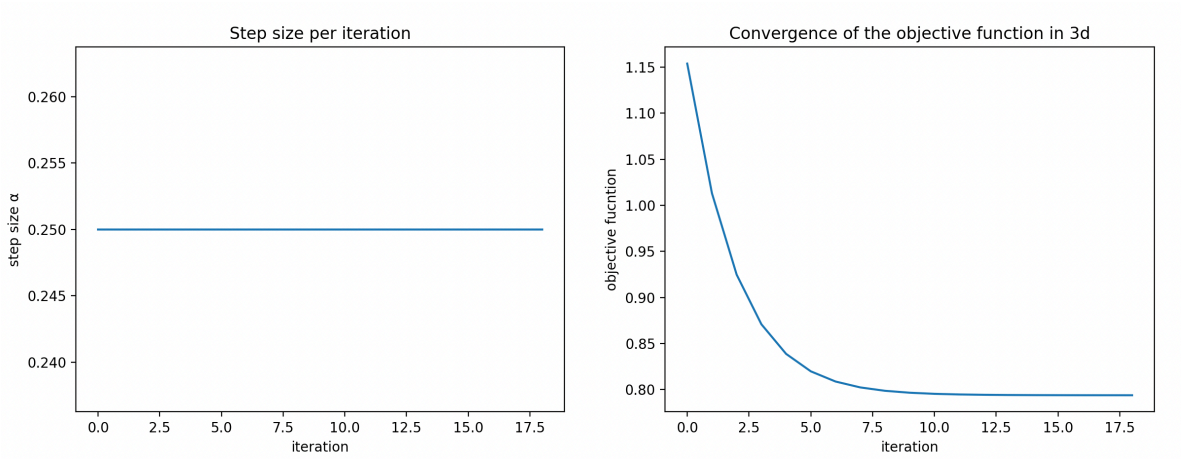

Figure 17: *Left*: Alpha returns during line-search newton method from Figure 16. *Right*: Convergence of the objective function from newton method from Figure 16

Discussion

The gradient descent and Newton methods were both applied to the log-barrier problem in three dimensions, under one and two half-space constraints respectively. In both configurations, both methods successfully converged to a feasible center point, with the Gradient descent method converging in fewer iterations. Interestingly the newton method implementation didn't seem to use the line-search. This might be cause it uses the hessian scale the steps. For convex functions, theory would suggest that newtons methods should converge in fewer steps than the gradient descent. This could be due to the clipping mechanic of the code, interfering with the newtons ability to find the center point. Given more time, with this project, this would be researched more in detail, and perhaps modify the line search code to modify the step size to better account for leaving the feasible set.

Conclusion on case 1

The log-barrier is successfully used to define the a feasible center of the convex polyhedron, with the minimize lying strictly within P. Convexity of the barrier function was proven both through the composition of convex and affine function, but also through positive definiteness of the Hessian.

Both gradient descent and Newton's method were applied to the problem in two and three dimensions with one and two inequality constraints. Both methods converged reliably to feasible center points. While both implementations gave good results, the gradient descent proved to perform better in terms of iterations, but this is most likely due to an error in the implementation.

Case 2

In this case the objective is to fairly distributes students across teams such that their attributes match the most. As each attribute carry a weight to determine its importance.

Task 1

This can be modeled as a Integer Linear Programming (ILP) problem.

Given the objective function

$$\sum_{T\in\mathcal{T}}\sum_{a\in A} w(a)z_{T,a} = \sum_{T\in\mathcal{T}}\sum_{a\in A}\sum_{b\in L_a} w(a)y_{T,a,b} \tag{44}$$

Then task is to minimize $\sum_{T\in\mathcal{T}}\sum_{a\in A}\sum_{b\in L_a} w(a)y_{T,a,b}$

Lets go over the different constraints for this problem
1. Each student can only be assigned to exactly 1 team

$$\sum_{T\in\mathcal{T}} x_{s,T} = 1 \quad \forall s \in S \tag{45}$$

2. The size of a team is limited to $\ell \leq |T| \leq u$

$$\ell \leq \sum_{s\in S} x_{s,T} \leq u \quad \forall T \in \mathcal{T} \tag{46}$$

3. Two student who share a disagreement, must not be on the same team

$$x_{s_1,T} + x_{s_2,T} \leq 1 \quad \forall (s_1, s_2) \in D, \forall T \in \mathcal{T} \tag{47}$$

And so one can define the minimization task as the following:

$$\begin{aligned} \min \sum_{T\in\mathcal{T}}\sum_{a\in A}\sum_{b\in L_a} w(a)y_{T,a,b} \\ s.t \quad \sum_{T\in\mathcal{T}} x_s = 1 \quad \forall s \in S \\ \ell \leq \sum_{s\in S} x_s \leq u \quad \forall T \in \mathcal{T} \\ x_{s_1,T} + x_{s_2,T} \leq 1 \quad \forall (s_1, s_2) \in D, \forall T \in \mathcal{T} \\ x_{s,T}, y_{T,a,b} \in \{0, 1\} \end{aligned} \tag{48}$$

Task 2, 3 and 4 all implement the `ROAR-NET-API`. This allows using highly optimized construction and search functions like `greedy_construction` and `beam_search`. Since `ROAR-NET-API` is an external framework, is expects a problem to implement certain interfaces like: `SupportsLowerBound` or `SupportsApplyMove`. These interfaces define how the framework can interact with the problem without knowing the specific problem details.

By using `ROAR-NET-API` one can focus on implementing classes for the Problem, Solution, Move, and Neighbourhood, while the framework handles the search process. Because of this the development process is split into 3 tasks.
1. Solution construction (Task 2)
2. Solution improvement (Task 3)
3. Meta-heuristics (Task 4)

Task 2 - construction

Lets define problem and implement a solution construction, such that the `ROAR-NET-API` can use it.

This is done by
1. Defining the problem
2. Define the solution
3. Search for valid moves
4. Initialize the problem iteratively with valid moves

The problem needs a class for defining the problem itself.

```python
class Problem(...):
    def __init__(...):
        self.n_members = n_members
        self.n_teams = n_teams
        self.weights = weights
        self.attributes = attributes
        self.disagreements = disagreements
        self.min_size = min_size
        self.max_size = max_size
```

Code snippet 12: Part of the `Problem` implementation which implements how a problem is defined for this optimization task.

The problem (Code snippet 12) should by use of the method `from_textIO(cls, f: textIO)` translate the `instance` to a problem which is readable from the `ROAR-NET-API`.

The problem should moreover initialize the both the neighbourhood and lists a list of sets to map disagreements for each student. This is shown on Code snippet 13. This way the algorithm can always see for any student which other students are in disagreements.

```python
        self.disagrees_with: list[set[int]] = [set() for _ in range(n_members)]
        for a, b in disagreements:
            self.disagrees_with[a].add(b)
            self.disagrees_with[b].add(a)
```

Code snippet 13: Part of the `Problem` implementation, this part creates the `disagrees_with`

After the problem definition, its also important that the solution is defined. The solution should contain: a reference to the problem instance (so the solution can access problem data), a list of assignments, the labels within each team in the solution, the team sizes, and the lower bound.

```python
class Solution(SupportsCopySolution, SupportsLowerBound):
  def __init__(
      self,
      problem: "Problem",
      assignments: list[int],
      team_labels: list[list[dict[int, int]]],
      team_sizes: list[int],
      lb: int,
  ):

  def objective_value(...):
  def lower_bound(...):
  def copy_solution(...):
  def is_feasible(...):
  def __repr__(...):
```

Code snippet 14: The `Solution` class defines what a solution should hold such that methods can make assignments through the optimization process.

The solution class (Code snippet 14) also includes a `objective_value` method, a `lower_bound` method, a `copy_solution` method and a `is_feasible` method. These will all be useful later on, since they are used either in testing or for some of the solvers. Now that the problem is well defined, one can create an empty solution. The `empty_solution` is then just an instance of a solution with empty teams and a lower bound of 0.

Now one can represent the problem as a solution (though an empty one) by `print(Problem.from_textIO(file).empty_solution())`

```
output
Solution(
  assignments=[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
  team_labels=[[{}, {}, {}], [{}, {}, {}], [{}, {}, {}]],
  team_sizes=[0, 0, 0],
  lb=0
)
```

Code snippet 15: Empty constructed solution given 13 students and 3 teams

Notice how on Code snippet 15, team assignments are labeled as `-1`, meaning that all students are unassigned.

For the improvement task, its smart construct a feasible solution. This solution might be suboptimal, but it might work better for most improvement methods, rather than a unsigned (unfeasible) solution. For this any of the construction algorithms from `roar_net_api.algorithms` may be used. The `AddNeighbourhood` class (Code snippet 16) should therefore create a neighborhood of moves for such an algorithm.

```python
Python
class AddNeighbourhood(SupportsMoves[Solution, AddMove]):
    def __init__(self, problem: "Problem", neighbourhoodType=None):
        self.problem = problem
        self.neighbourhoodType = neighbourhoodType

    def moves(...)
```

Code snippet 16: The `AddNeighbourhood` keeps track of the problem, and which neighborhood type is being used at runtime. This will be elaborated later on Code snippet 17

This `moves()` method is only used while **constructing** the initial solution, because of this, the moves available to the construction algorithms provided by the framework should only consider moves for students which are unassigned. The improvement where students are moved comes later.

After this, it may be beneficial to select candidate moves using some additional selection criteria. Initially, the construction can be performed by identifying all feasible moves and greedily selecting the best move at each step. Instead of acting as a traditional heuristic that merely prioritizes moves, the approach used here is closer to neighborhood pruning. Rather than exploring the complete set of feasible moves, only a reduced subset of the neighborhood is considered. The excluded moves are those which are believed to have a low likelihood of improving the situation. Note that although these assumptions are not formally provable (as believed by the authors), it might either reduce the computational time spent constructing by avoiding exploration of moves that are unlikely to be good.

Lets examine four examples of limiting the neighbourhood.

1. Restrict to moves that fill smaller teams.
2. Restrict to moves that assigns 'most disagreeing' members
3. Hybrid
4. All feasible moves

For testing efficiency, the `AddNeighbourhood` class implements all four, with a match-case IN THE `moves` method shown on Code snippet 17.

```python
def moves(self, solution: Solution) -> Iterable[AddMove]:
        match (self.neighbourhoodType):
            case "minFirst":
                yield from self.minFirst(solution) 1

            case "mostDisagreementsFirst":
                yield from self.mostDisagreementsFirst(solution) 2

            case "hybridMoves":
                yield from self.hybridMoves(solution) 3

            case "allFeasible":
                yield from self.allFeasible(solution) 4
```

Code snippet 17: The `moves` method, which selects which kind of move filtering to use. This is mostly good implantation practice for easier testing.

All move-methods follow a similar structure of: (1) identify which students to be assigned (those who are unassigned), (2) candidate move filtering, (3) add the moves which are feasible.

```python
        p = self.problem
        unassigned = [s for s, team in enumerate(solution.assignments)
                      if team == -1]
        if not unassigned:
            return # if no un assigned student, early exit
```

Code snippet 18: Part of a generic `moves` which explains the common behavior of the moves methods implemented later on. Here all the unassigned student are selected before any filtering.

As shown in Code snippet 18 Every move in the construction phase needs to only assign students whom are unassigned.

```python

         for s in unassigned:
             for t in candidate_teams:
                 if self.is_feasible_add(s, t, solution):
                     yield AddMove(s, t)
```

Code snippet 19: Part of a generic `moves` which explains the common behavior of the moves methods implemented later on. Here all the feasible moves for $s \in \mathcal{S}_{\text{unassigned}}$ and $T \in \mathcal{T}_{\text{candidates}}$ are yielded such that they can be applied by a solver.

Furthermore as shown on (Code snippet 19), for each student, the move should only yield moves which a permitted under the disagreement rule. By searching blocked teams for a student, allows choosing which teams are blocked due to this rule. The it checks if the a move is feasible before yielding it. Lets now discuss the different kinds of move filtering. For the following methods repeating code is commented out since its the same, unless specified.

All feasible moves

One could use the `allFeasible` (Code snippet 20) which captures all feasible moves within the constraints of the problem.

```python
def allFeasible(self, solution: Solution) -> Iterable[AddMove]
    # identification of unassigned students

    under_min = [t for t in all_teams if solution.team_sizes[t] < p.min_size]
    if under_min:
            candidate_teams = under_min
        else:
            candidate_teams = [t for t in all_teams if solution.team_sizes[t] < p.max_size]

# yield all the legal moves
```

Code snippet 20: The `allFeasible` method yield all feasible moves to a solver.

Restrict to smallest teams

```python
def minFirst(self, solution: Solution) -> Iterable[AddMove]
    # identification of unassigned students

    smallest = min(solution.team_sizes[t] for t in all_teams)
    candidate_teams = [t for t in all_teams
                       if solution.team_sizes[t] == smallest]

    # yield all the legal moves
```

Code snippet 21: The `minFirst` method filters moves, such that assignments are only made to the teams with the least assignments (smallest teams) with the most disagreements are assigned first.

The `minFirst` (Code snippet 21) selects the minimum size out of all possible teams. Then selects the candidate teams as any team which is as small as the smallest team. This should create a balance in the team sizes, that ensures that the smallest teams are prioritized and filled first.

Restrict to most disagreeing members

In most assignments tasks, it is often wise to assign the more complex pieces first. The same method might be wise to approach this problem with.

```python
def mostDisagreementsFirst(self, solution: Solution) -> Iterable[AddMove]
    # identification of unassigned students

    max_dis = max(len(p.disagrees_with[s])for s in unassigned)
    unassigned_priority_students = [s for s in unassigned
                                    if len(p.disagrees_with[s]) == max_dis]

    candidate_teams = [t for t in all_teams
                       if solution.team_sizes[t] < p.max_size]


    for s in unassigned_priority_students:
    # yield all the legal moves
```

Code snippet 22: The `mostDisagreementsFirst` method filters moves which assigns students with less disagreements, such that students with the most disagreements are assigned first.

The `mostDisagreementsFirst` (Code snippet 22) method counts the maximum number of disagreements for the set of unassigned students. This way it can prioritize those whom enforce more strict requirements for assignments. By only selecting those with the most disagreements first, moves later on will be more feasible.

Hybrid of restricting heuristics

While `minFirst` and `mostDisagreementsFirst` each offer distinct advantages, a hybrid approach may capture the benefits of both.

```python
def hybridMoves(self, solution: Solution) -> Iterable[AddMove]
    # identification of unassigned students

    smallest = min(solution.team_sizes[t] for t in all_teams)
    candidate_teams = [t for t in all_teams
                       if solution.team_sizes[t] == smallest]

        if not candidate_teams:
            candidate_teams = [t for t in all_teams
                               if solution.team_sizes[t] < p.max_size]

    max_dis = max(len(p.disagrees_with[s])for s in unassigned)
    unassigned_priority_students = [s for s in unassigned
                                    if len(p.disagrees_with[s]) == max_dis]

    for s in unassigned_priority_students:
    # yield all the legal moves
```

Code snippet 23: The `allFeasible` method is a hybrid version of both the `mostDisagreementsFirst` method and the `minFirst` method. Much like `mostDisagreementsFirst` and `minFirst` it too filters the moves for the solver.

The `hybridMoves` (Code snippet 23) prioritizes both small teams, **and** disagreeing students. However more constraint might not always better as it might limit restrict the ability to explore more beneficial options in the search space.

Moving and incremation

Up until now, the lower bound has been incremented as

$$\Delta lb(s) = f(s) + h(s) \quad \text{where } h(s) = 0 \tag{49}$$

where $f(s)$ is the objective contribution of the move i.e. the weighted sum of new labels introduced and $h(s)$ is a heuristic term estimating future cost. Although a heuristic term $h(s)$ could be used to approximate future cost effects, this is not used as the complexity is not really worth it, given the problem difficulty. Instead, the simpler evaluation based solely on $f(s)$ is good enough for the construction process. This in incremented when a move is applied.

```python
class AddMove(...):
    def __init__(self, student: int, toTeam: int):
        self.s = student
        self.toTeam = toTeam

    def apply_move(self, solution: Solution) -> Solution:
        solution.lb += self.lower_bound_increment(solution)
        solution.assignments[self.s] = self.toTeam
        solution.team_sizes[self.toTeam] += 1
        member_labels = solution.problem.attributes[self.s]
        for a, label in enumerate(member_labels):
            counts = solution.team_labels[self.toTeam][a]
            counts[label] = counts.get(label, 0) + 1
        return solution
```

Code snippet 24: `AddMove` class and `apply_move` method implementation, which serves to define what move is, and how it should be applied to a given solution

Each `addMove` (Code snippet 24-4) should then keep track of its student and the team. It should then apply the move by assigning the student to the given team and updating that teams labels.

Any move should increment the lower-bound `lb` for the move such that the objective function can be minimized.

```python
    def lower_bound_increment(self, solution: Solution) -> float:
        p = solution.problem
        labels_count = solution.team_labels[self.toTeam]
        student_labels = p.attributes[self.s]
        incr = 0
        for a, label in enumerate(student_labels):
            if labels_count[a].get(label, 0) == 0:
                incr += p.weights[a]
        return incr
```

Code snippet 25: `lower_bound_increment` implementation, which serves to return the increment of the lower-bound given a certain move.

The increment (Code snippet 25) simply counts the weights of the labels that are absent if the move is made. This way one can determine if a move will be good/bad depending on the `lb`.

Testing & comparison

Lets test these against each other with larger instances of data (300 students). Since *lb* values depend on the heuristic used, they are not directly comparable across runs; instead, the objective value (cost) of the constructed solution is used for comparison. For this both the `greedy_search` and the `beam_search` are used. The `greedy_search` algorithm is a greedy construction algorithm which at each iteration picks the best-scoring move available, repeating until no more moves are available. The main drawback of using a greedy algorithm like `greedy_search` is that it only expands one single solution. The `beam_search` is much more sophisticated in that sense, since it expands multiple solutions simultaneously, and keeps the k best ones. This could make it very powerful when there are many different moves like in this problem.

Restricting heuristic : greedy_search	**cost**	**Time (sec)**
minFirst	70510	≈ 0.850
mostDisagreementsFirst	73590	≈ 0.536
hybridMoves	70960	≈ 0.540
allFeasible	68770	≈ 0.997
Restricting heuristic : beam_search - initial: random	**cost**	**Time (sec)**
minFirst	104670	≈ 0.002
mostDisagreementsFirst	104700	≈ 0.002
hybridMoves	105530	≈ 0.002
allFeasible	105170	≈ 0.002

Table 1: Construction heuristics comparison on 300 students / 60 teams.

Greedy construction consistently outperforms beam search across all tests (Table 1), achieving costs in the 68770–73590 range compared to 104670–105530 for beam search. This is probably explained by the tie-breaking issue: on an empty solution, beam search breaks on ties which is everywhere on an initial empty solution. One can avoid this with a random initialized solution but introduces a poor initial cost that beam search cannot recover from

Lets see how the best constructed solution looks so far without any improvement yet. This example (Code snippet 26) combines the best results from earlier.

```
Solution(                                                                output
Solution(
    assignments=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 5, 12, 13, 14, 14,
    15, 16, 17, 4, 18, 19, 20, 21, 22, 23, 24, 25, 26, 7, 27, 17, 28, 29,
    30, 6, 31, 32, 33, 22, 11, 0, 34, 32, 35, 17, 27, 36, 7, 14, 5, 7, 33,
    30, 0, 15, 34, 16, 28, 12, 35, 37, 0, 38, 22, 38, 23, 37, 8, 39, 40,
    41, 42, 38, 43, 11, 44, 39, 45, 42, 25, 46, 23, 23, 9, 8, 47, 45, 48,
    15, 36, 31, 2, 9, 45, 49, 25, 47, 25, 43, 12, 49, 44, 27, 19, 28, 4,
    45, 38, 13, 50, 20, 13, 25, 50, 48, 51, 52, 18, 46, 53, 1, 54, 55, 48,
    45, 56, 31, 8, 1, 56, 41, 40, 24, 10, 35, 30, 26, 47, 35, 34, 1, 57,
    55, 9, 48, 30, 19, 16, 17, 44, 15, 49, 51, 3, 7, 54, 41, 21, 46, 6, 9,
    26, 46, 55, 29, 34, 13, 32, 26, 30, 21, 46, 12, 6, 36, 10, 12, 40, 35,
    18, 5, 11, 58, 52, 40, 37, 39, 52, 51, 15, 24, 59, 57, 2, 29, 52, 3,
    54, 20, 55, 49, 11, 51, 18, 14, 16, 18, 33, 58, 58, 6, 53, 42, 22, 50,
    57, 3, 41, 24, 27, 5, 17, 26, 19, 50, 56, 4, 55, 59, 53, 29, 44, 47,
    27, 34, 54, 8, 39, 4, 44, 43, 33, 33, 31, 59, 20, 59, 32, 50, 36, 53,
    28, 54, 42, 38, 49, 32, 48, 10, 21, 22, 37, 58, 43, 2, 31, 57, 47, 59,
    43, 14, 39, 51, 53, 20, 28, 42, 40, 57, 52, 56, 0, 2, 56, 41, 10, 1,
    3, 13, 21, 36, 19, 37, 24, 29, 23, 16, 58],
    team_labels=[[{0: 5}, {3: 5}, {3: 2, 5: 3}, ..., {7: 1, 4: 1, 1: 1, 2:
    1, 0: 1}]],
    team_sizes=[5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,
    5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,
    5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5],
    cost=cost=68770
)
Execution time: 0.9978528022766113
Is feasible: True
```

Code snippet 26: This is the best feasible construction solution using the (*note that team_labels has been reduced, because its very long for 300 student*)

Task 3 - improvement

Now its time to implement some solution improvement methods, such that the `ROAR-NET-API` can use it. Since the algorithms are now improving an already constructed solution, they should move students around rather than add the. The goal in improvement is so make moves that minimizes the `objective_value` as mush as possible.

This is done by
3. Search for valid moves
4. Initialize the problem iteratively with valid moves

Improving a solution can be done in several ways. Here, swap moves are used — exchanging two students between teams — as the local search neighbourhood. The solvers, which are to solve given the following methods, should then be presented a set of moves, and the ability to see the objective value increment of each move. See for example the `best_improvement` method from the `ROAR-NET-API`, which does exactly this

```python
    for move in neigh.moves(solution):
        incr = move.objective_value_increment(solution)
        assert incr is not None
        if incr < 0:
            yield (move, incr)
```

Code snippet 27: `ROAR-NET-API` implementation of `best_improvement` which utilizes the `move.objective_value_increment(solution)` to select moves

Random sampling heuristic

Firstly one can still change the search space, by for example implementing a random moves. This will serve as an option to a possible heuristic based algorithm like SA to explore non-improving moves unpredictably. The following `randomMoves` is implemented in the `SwapNeighbourhood` class, however a method like this will be implemented for all neighbourhood classes.

```python
    def random_move(self, solution: Solution) -> SwapMove | None:
        return next(self.random_moves_without_replacement(solution), None)

    def random_moves_without_replacement(self, solution: Solution) -> Iterable[SwapMove]:
        p = self.problem
        n = p.n_students
        total = n * n

        for idx in sparse_fisher_yates_iter(total):
            s1 = idx // n
            s2 = idx % n
            if s1 >= s2:
                continue

            t1, t2 = solution.assignments[s1], solution.assignments[s2]

            if t1 == t2:
                continue
            if self._is_feasible_swap(s1, s2, t1, t2, solution):
                yield SwapMove(s1, s2, t1, t2)
```

Code snippet 28: `random_move` + `random_moves_without_replacement` method implemented to be used by improvement methods which deliberately takes worse choices to escape local optima's

The `random_move` (Code snippet 28) iterates over the ordered pairs (s_1, s_2) where $s_1 \neq s_2$ as given from the `sparse_fisher_yates_iter` method. This way the `random_moves_without_replacement` method can decode each iteration back to two different students. This works since the shuffle works over the total of $|S| \cdot |S|$. Roughly half the indices are discarded (where s1 ≥ s2), so there might exist some more efficient methods of encoding, but the simplicity of this approach works just fine for this project.

```python
def sparse_fisher_yates_iter(n: int) -> Iterable[int]:
    p: dict[int, int] = dict()
    for i in range(n - 1, -1, -1):
        r = random.randrange(i + 1)
        yield p.get(r, r)
        if i != r:
            p[r] = p.get(i, i)
```

Code snippet 29: The `sparse_fisher_yates_iter` method implemented for effective shuffling

The `sparse_fisher_yates_iter` (Code snippet 29) method is used here to shuffle the set of teams, such that the `random_moves_without_replacement` method can randomly assign students to teams. It does this by randomly selecting a number from the range `n-1`, keeping track of seen numbers with a `dict()`

Swap team moves

The following methods should implement the behavior of two student s_1 and s_2 swapping teams T_1 and T_2. More formally

$$m(s, t_{\text{from}}, t_{\text{to}}) : \sigma(s) = t_{\text{from}} \rightarrow \sigma(s) = t_{\text{to}} \tag{50}$$

```python
class SwapMove(...):
    def __init__(self, student1: int, student2: int, team1: int, team2: int):
        self.s1 = student1
        self.s2 = student2
        self.t1 = team1
        self.t2 = team2

    def apply_move(self, solution: Solution) -> Solution:
        prob = solution.problem
        solution.lb += self.objective_value_increment(solution)

        # s1 leaves t1, joins t2
        for a, label in enumerate(prob.attributes[self.s1]):
            from_counts = solution.team_labels[self.t1][a]
            from_counts[label] -= 1
            if from_counts[label] == 0:
                del from_counts[label]
            to_counts = solution.team_labels[self.t2][a]
            to_counts[label] = to_counts.get(label, 0) + 1
        solution.assignments[self.s1] = self.t2
```

Code snippet 30: Part of the `apply_move` method that re-assigns student 1 s_1 to their new team

Firstly s_1 leaves T_1 and joins T_2, such that both teams are updated. Here their label-count is updated trough the references `from_counts` and `from_counts` corresponding to the team s_1 is coming from and going to. Then in the end, the actual assignment is made

```python
        # s2 leaves t2, joins t1
        for a, label in enumerate(prob.attributes[self.s2]):
            from_counts = solution.team_labels[self.t2][a]
            from_counts[label] -= 1
            if from_counts[label] == 0:
                del from_counts[label]
            to_counts = solution.team_labels[self.t1][a]
            to_counts[label] = to_counts.get(label, 0) + 1
        solution.assignments[self.s2] = self.t1
        return solution
```

Code snippet 31: Part of the `apply_move` method that re-assigns student 2 s_2 to their new team

The exact same is done to s_2 (Code snippet 31).

The increment (Code snippet 32) should then also resemble the swap such that, its updated with respect the the weights and labels in the two teams

```python
def score_increment(self, solution: Solution) -> float:
        prob = solution.problem
        from_counts = solution.team_labels[self.fromTeam]
        to_counts = solution.team_labels[self.toTeam]
        s_labels = prob.attributes[self.s]
        incr = 0
        for a, label in enumerate(s_labels):
            # s leaves fromTeam: lb drops if s was the last holder of this label
            if from_counts[a].get(label, 0) == 1:
                incr -= prob.weights[a]
            # s joins toTeam: lb rises if toTeam has no holder of this label yet
            if to_counts[a].get(label, 0) == 0:
                incr += prob.weights[a]
        return incr
```

Code snippet 32: Score incrementation implemented for the `SwapMove` method

For task 3 the constructed solutions from task 2 is tested and improved upon on 2 different solvers, that is the `best_improvement` and the `first_improvement`.

Testing and comparison

Now lets some different improvement algorithms on this swap move. This test will include `best_improvement` and `first_improvement` These algorithms are simple local search heuristics that serve as useful baselines before introducing more advanced meta-heuristics later (Task 4). The `first_improvement` algorithm applies the first improving move it encounters, while `best_improvement` evaluates all improving moves and selects the best one. This will compare on the `greedy_construction` from earlier since, it proved itself most competitive.

```
Is feasible: True                                                    output
Solution(
  cost=68770
)
Execution time: 0.996727705001831
Is feasible: True
Solution(
  cost=7240
)
Execution time: 22.735021829605103
```

Code snippet 33: Output of the `best_improvement` method improving on the `greedy_construction` method on 300 students

The `best_improvement` (Code snippet 33) is a remarkable improvement of almost $\approx 90\%$ lower objective value. Though it came with a cost of almost 23 seconds to compute. Note that `best_improvement` is $O(n^2)$ so given this large instance of 300 students that ≈ 90000 indices pr. iteration. because of this it's expected and acceptable that it takes some time. Lets take a look at the first improvement.

```
Is feasible: True                                                    output
Solution(
  cost=68770
)
Execution time: 0.9894468784332275
Is feasible: True
Solution(
  cost=7430
)
Execution time: 1.604125738143921
```

Code snippet 34: Output of the `first_improvement` method improving on the `greedy_construction` method on 300 students

The `first_improvement` (Code snippet 34) did almost the same improvement of $\approx 89\%$ lower objective value in roughly $\approx 6\%$ of the time. This suggests that the neighbourhood in this instance is relatively "easy" in the sense that good improving moves are found quickly, often early in the neighbourhood. In such a setting, first-improvement benefits from avoiding a full neighbourhood evaluation, while still reaching comparable local optima. In contrast, `best_improvement` is more sensitive to this structure, as it evaluates all moves even when good improvements are already available early.

Task 4 - meta-heuristics

For this task, several meta-heuristic algorithms could be explored. Simulated Annealing (SA) is selected here, as it is good for escaping local optima by probabilistically accepting worsening moves, and so makes it well suited for this problem. Besides this it creates the opportunity to experiment with cooling schedules and their effect on solution quality, which the group finds of much interest.

The SA uses the Metropolis condition

$$p = \exp\left(\frac{-\Delta}{T_i}\right) \tag{51}$$

Where Δ is the cost increase, and T_i is the current temperature. This criteria ensures a move that worsens the solution by Δ is accepted with a probability of $\exp\left(\frac{-\Delta}{T_i}\right)$. Then for the initial temperature, one can just fix a probability of 50%. Δ can be approximated by running a series objective values on some of random moves.

```python
def estimate_delta(neighbourhood, solution, n_samples) -> float:
    deltas = []
    for _ in range(n_samples):
        move = neighbourhood.random_move(solution)
        if move is None:
            break
        incr = move.objective_value_increment(solution)
        print(incr)
        if incr > 0:
            deltas.append(incr)
    return sum(deltas) / len(deltas) if deltas else 1.0
```

Code snippet 35: Estimating the average deltas of the move increment of 100 random moves

With a quick test (Code snippet 35) the average delta of 100 random iterations is set to 1. This means that before SA even begins there a almost only acceptable moves, because the

Then T_0 can be calculated as

$$T_0 = \frac{-1}{\ln(0.5)} \approx 3.32 \tag{52}$$

Once this is done, the different temperature schedulers can be explored. Lets look at `geometric` and `cosineRestarts` (Zhang 2026)

Geometric decay scheduler

The geometric decay scheduler (Code snippet 36), also known as factor decay, applies a constant decay factor at each iteration making a polynomial smooth decay.

$$T_{i+1} = \alpha \cdot T_i \quad \text{where } \alpha \in [0, 1] \tag{53}$$

```python
def geometric(t0: float, cooling: float = 0.999) -> Callable[[int], float]:
    def schedule(k: int) -> float:
        return t0 * (cooling ** k)
    return schedule
```

Code snippet 36: Geometric (factor) decay implemented for the simulated annealing method

Cosine scheduler

Now to implement a completely different decay scheduler; The Cosine Decay.

$$T_i = T_{\min} + \frac{T_0 - T_{\min}}{2}\left(1 \cdot \cos\left(\frac{\pi i}{i_{\max}}\right)\right) \quad \text{where } i = 1, 2, ...n \text{ iterations.} \tag{54}$$

The cosine decay scheduler (Code snippet 37) follows the same formular as presented above, decaying from an initial temperature t_0 down to the minimum temperature $t_{\min}$ with a cosine curve. This can be favorable since is spends more iterations being warm, and only cooling at the end. This supports the procces that the most important places to allow worse decisions might be in the beginning.

```python
def cosine(t0: float, t_min: float = 1.0, total: int = 1_000_000) ->
                                            Callable[[int], float]:
    def schedule(k: int) -> float:
        progress = min(k / total, 1.0)
        return t_min + 0.5 * (t0 - t_min) * (1 + math.cos(math.pi *
        progress))
    return schedule
```

Code snippet 37: Cosine decay implemented for the simulated annealing method.

Simulated annealing

```python
def sa(problem, solution: Solution,budget: float = 60.0,
  p_accept: float = 0.5, scheduler: Callable[[int], float] = None,
  record: bool = False) -> tuple[Solution, list] | Solution:
  """
  Simulated Annealing (SA), using a scheduler, for the assignment
  problem implemented using `ROAR-NET-API`
  """

  neighbourhood = problem.local_neighbourhood()
  s = solution.copy_solution()
  best = s.copy_solution()

  delta = estimate_delta(neighbourhood, s)
  t0 = -delta / math.log(p_accept)
  if scheduler is None:
      scheduler = geometric(t0)

  history = []
  t_start = time.time()
  k = 0
  while time.time() - t_start < budget:
      move = neighbourhood.random_move(s)
      if move is None:
          break
      incr = move.objective_value_increment(s)
      temp = scheduler(k)
      if temp == 0:
          break
      accepted = incr < 0 or random.random() < math.exp(-incr / temp)
      if accepted:
          move.apply_move(s)
          if s.lb < best.lb:
              best = s.copy_solution()
      if record:
          history.append({"k": k, "temp": temp, "cost": s.lb})
      k += 1

  return (best, history) if record else best
```

The simulated annealing method above finds a best solution, within a time interval (constrained by the decay). It initiates a Δ value with the `estimate_delta`, and creates random moves within the time interval, selecting better moves along the way. The simulated annealing method escapes local optima by sometimes taking bad moves. This is done at line `28`, where a move is accepted if the increment is smaller (e.i. the move is better), or at random times given the scheduler allows it.

Testing

Now lets test both of the schedulers to see their performance on this problem.

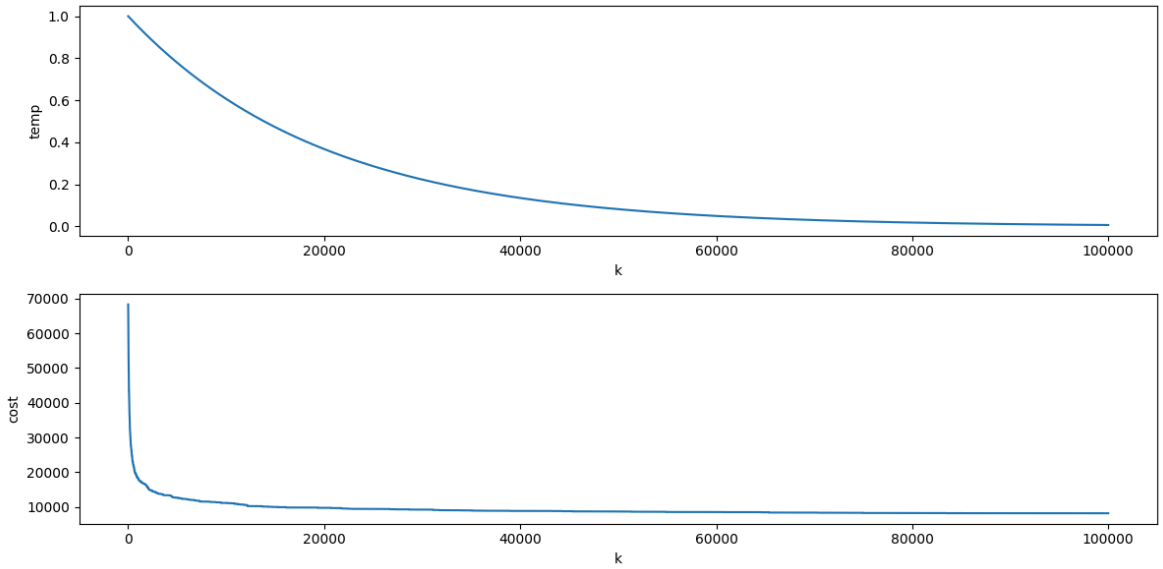

Figure 18: Here is the simulated annealing depicted with a geometric scheduler. The upper figure shows the temperature decreasing by a constant factor, and lower figure shows the cost dropping accordingly

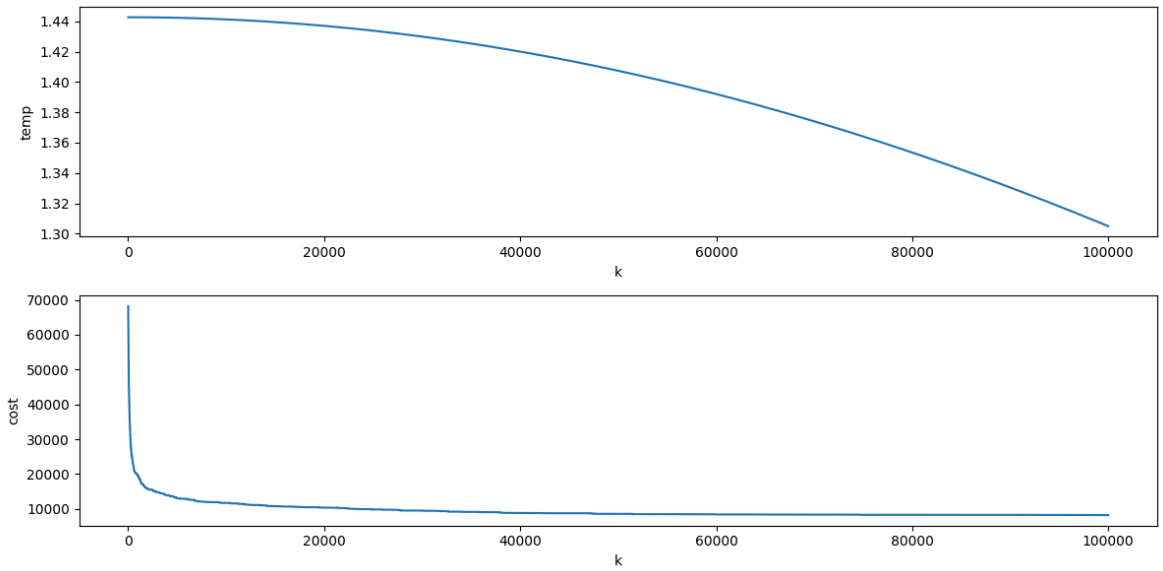

Figure 19: Here is the simulated annealing depicted with a cosine scheduler. The upper figure shows the temperature decreasing with a cosine shape, and the lower figure shows cost dropping accordingly

As shown on Figure 18 and Figure 19, both schedulers can be visualized to verify expected behavior. Figure 18 shows the geometric scheduler decaying rapidly toward zero, while Figure 19 shows the cosine scheduler cooling gradually.

For testing, one can do iterative restarts on the simulated annealing method. This means doing multiple runs of simulated annealing and updating the best of the solutions. In the following example, the simulated annealing algorithm was run **10 times**. because of this, the execution time also reflects this.

```
Is feasible: True                                                    output
Solution(
  cost=68770
)
Execution time: 1.0040440559387207
Is feasible: True
Solution(
  cost=6660
)
Execution time: 358.7554180622101
```

Code snippet 39: Solution made by simulated annealing with a geometric scheduler

```
Is feasible: True                                                    output
Solution(
  cost=68770
)
Execution time: 0.9936981201171875
Is feasible: True
Solution(
  cost=6800
)
Execution time: 622.6760261058807
```

Code snippet 40: Solution made by simulated annealing with a cosine scheduler

The simulated annealing methods (Code snippet 39 and Code snippet 40), improves even further on the solution, and results in a better solution than both the `best_improvement` and `first_improvement`. The best result of the simulated annealing methods resulted in a best solution of `cost=6660`. This came using the geometric scheduler. Note that all results from Case 2, are tested on a Mac Book Air 15, with an apple M4 chip and 16 GB ram.

Conclusion on Case 2

The results from the case 2 experiments has shown that the implemented methods to be a valuable the optimization process. The greedy construction heuristic reliably produces complete, feasible solutions. The local search improvement methods showed significantly better solutions compared to the constructed ones. The simulated annealing proved to be the most effective approach overall, being able to escape

local optima through somewhat controlled randomness. It achieved the overall lowest solution costs within what the authors view as an acceptable computational time. The best solution achieved with the simulated annealing using a geometric scheduler achieved the low cost of `6660`. The implemented methods overall each create a distinct role in the optimization process, and offers different trade-offs between solution quality and computational effort.

Bibliography

WrightS, Jorge Nocedal Stephen J. 2006. *Numerical Optimization, Second Edition*. Springer Series in Operations Research.

Zhang, Aston. 2026. "Dive into Deep Learning : Learning Rate Scheduling.". https://d2l.ai/chapter_optimization/lr-scheduler.html.